

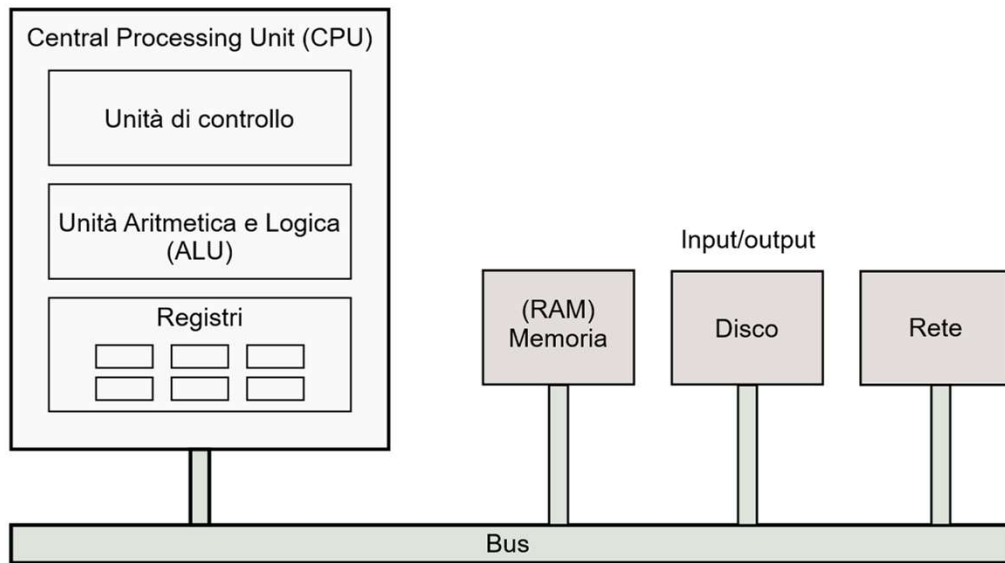
II Processore RISC-V

Obiettivo

Costruzione di una CPU RISC-V che sia in grado di eseguire:

- le istruzioni aritmetico-logiche di tipo R: add, sub, and, or
 - `add t0, t1, t2`
 - `sub t0, t1, t2`
- le istruzioni di riferimento alla memoria load word (lw) e store word (sw)
 - `lw t0, 12(t1)`
 - `sw t0, 12(t1)`
- le istruzioni di salto condizionato dal risultato di un test di uguaglianza
 - `beq t0, t1, etichetta`

Richiamo: Macchina di Von Neumann



- La CPU si compone di diverse parti distinte: **unità di controllo, unità aritmetico-logica, registri**
- I registri, l'unità aritmetico-logica e alcuni bus che li collegano compongono il **data path**
- Due registri importanti: Program Counter (PC) e Instruction Register (IR)
- La main memory contiene sia istruzioni sia dati usando sequenze di bit.

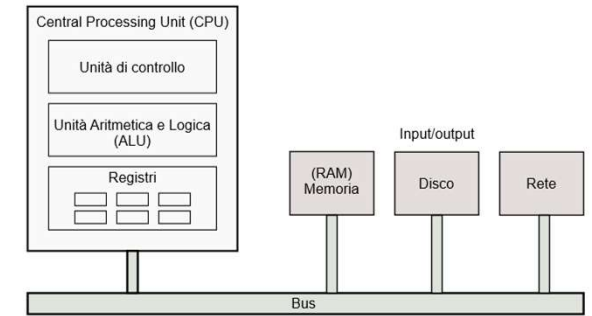
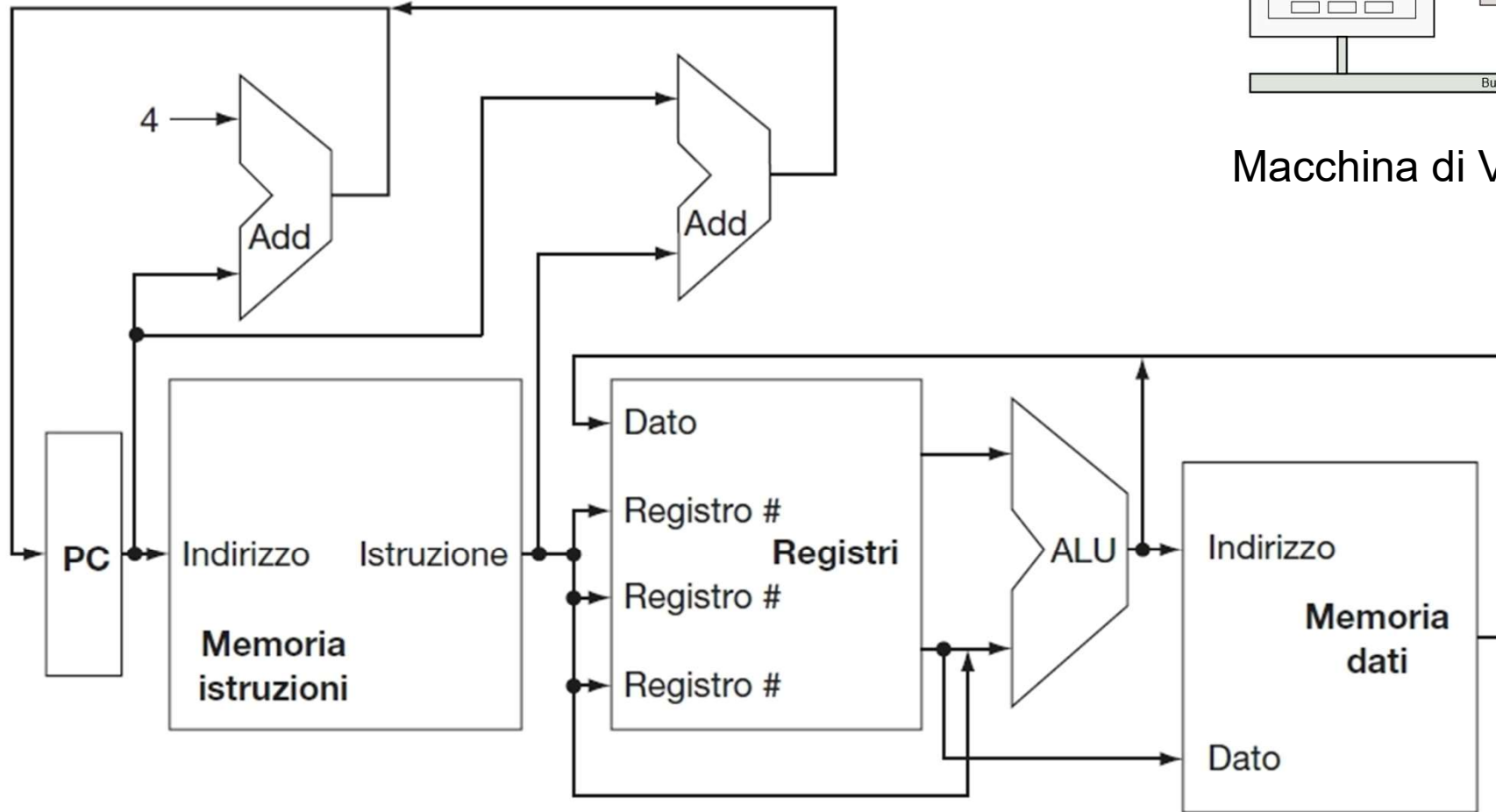
Ciclo del data path: processo di far passare due operandi attraverso la ALU e memorizzarne il risultato

Richiamo: Ciclo di Fetch-Decode-Execute

La CPU esegue ogni istruzione per mezzo di una serie di passi elementari:

1. Prendi l'istruzione seguente dalla memoria
2. Cambia il PC (program counter) per indicare l'istruzione seguente
3. Determina il tipo dell'istruzione appena letta
4. Se l'istruzione usa una parola in memoria, determina dove si trova
5. Metti la parola, se necessario, in un registro della CPU
6. Esegui l'istruzione
7. Torna al punto 1 e inizia a eseguire l'istruzione successiva

Data Path RISC-V (Semplificato)

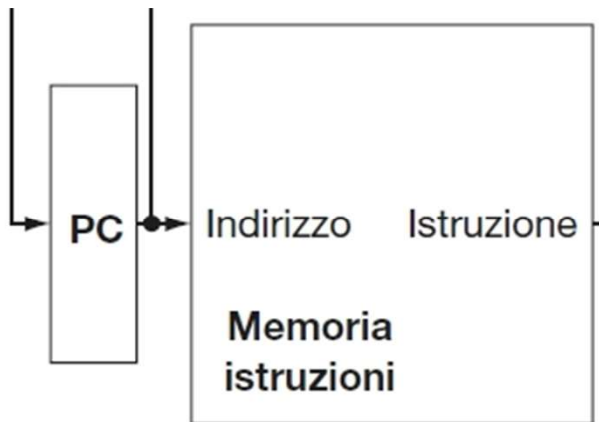


Macchina di Von Neumann

Data Path RISC-V (Semplificato)

(fetch)

- Il PC fornisce un indirizzo dalla memoria delle istruzioni
- Per semplificazione, assumiamo che la "memoria delle istruzioni" sia "read-only"
- Assumiamo anche che ci siano 2 memorie diverse:
Una per le istruzioni ed un'altra per i dati.



SP $\rightarrow$ 7FFF EFFF_{esa}

1001 0000_{esa}

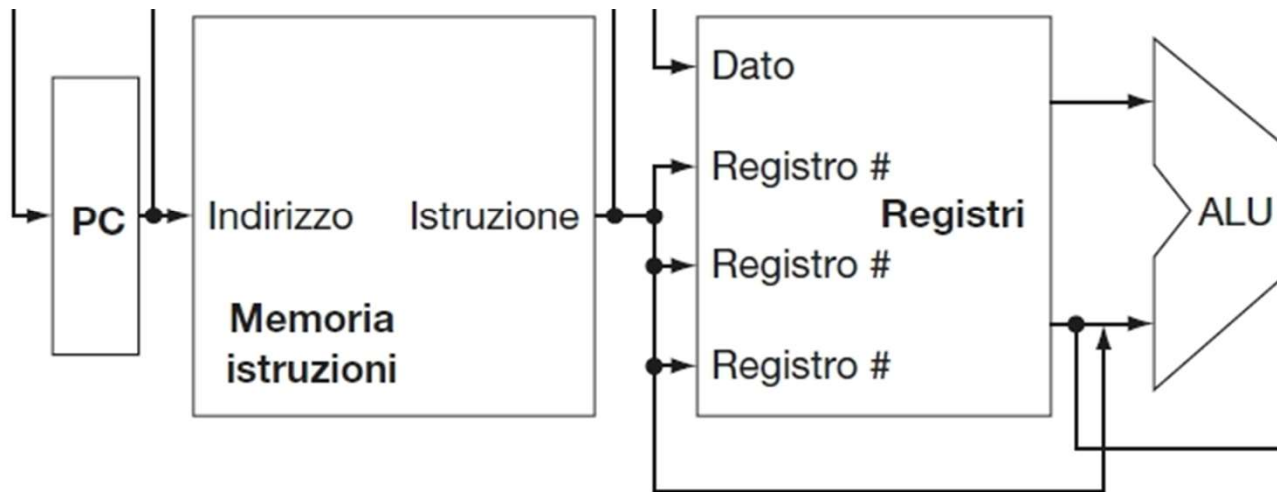
PC $\rightarrow$ 0040 0000_{esa}



Data Path RISC-V (Semplificato)

(decode)

- La ALU deve "capire" il tipo di istruzione
- Per esempio, il numero d'ordine dei registri contenenti gli operandi può essere letto nei campi opportuni dell'istruzione stessa
- **L'unità di controllo (non mostrata nella figura) è la parte che effettivamente decodifica le istruzioni**

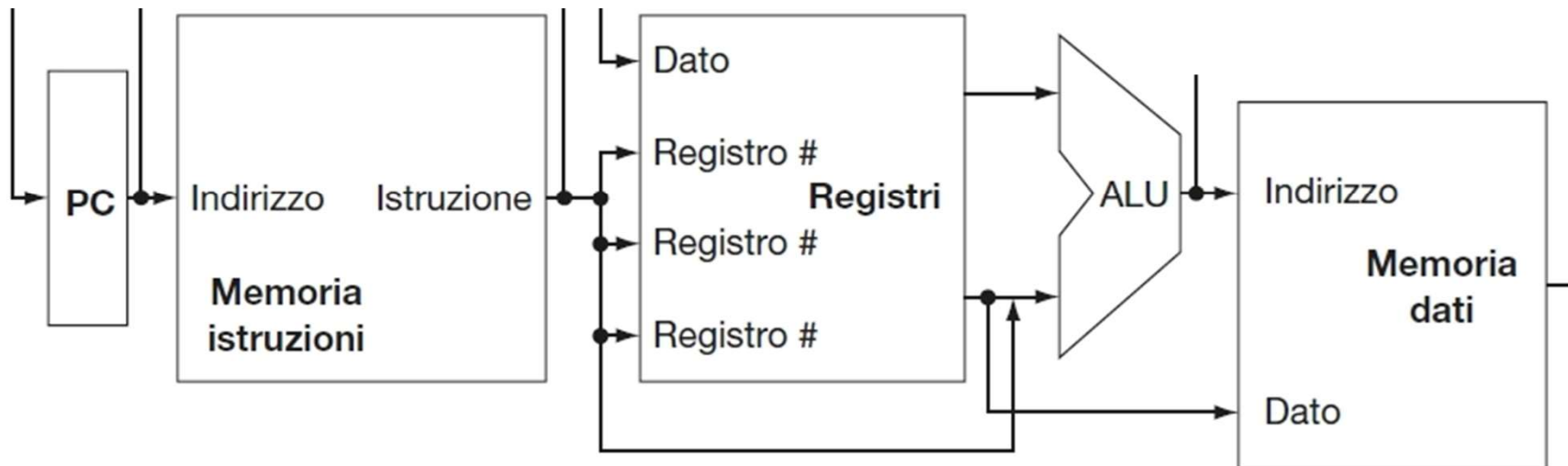


Data Path RISC-V (Semplificato)

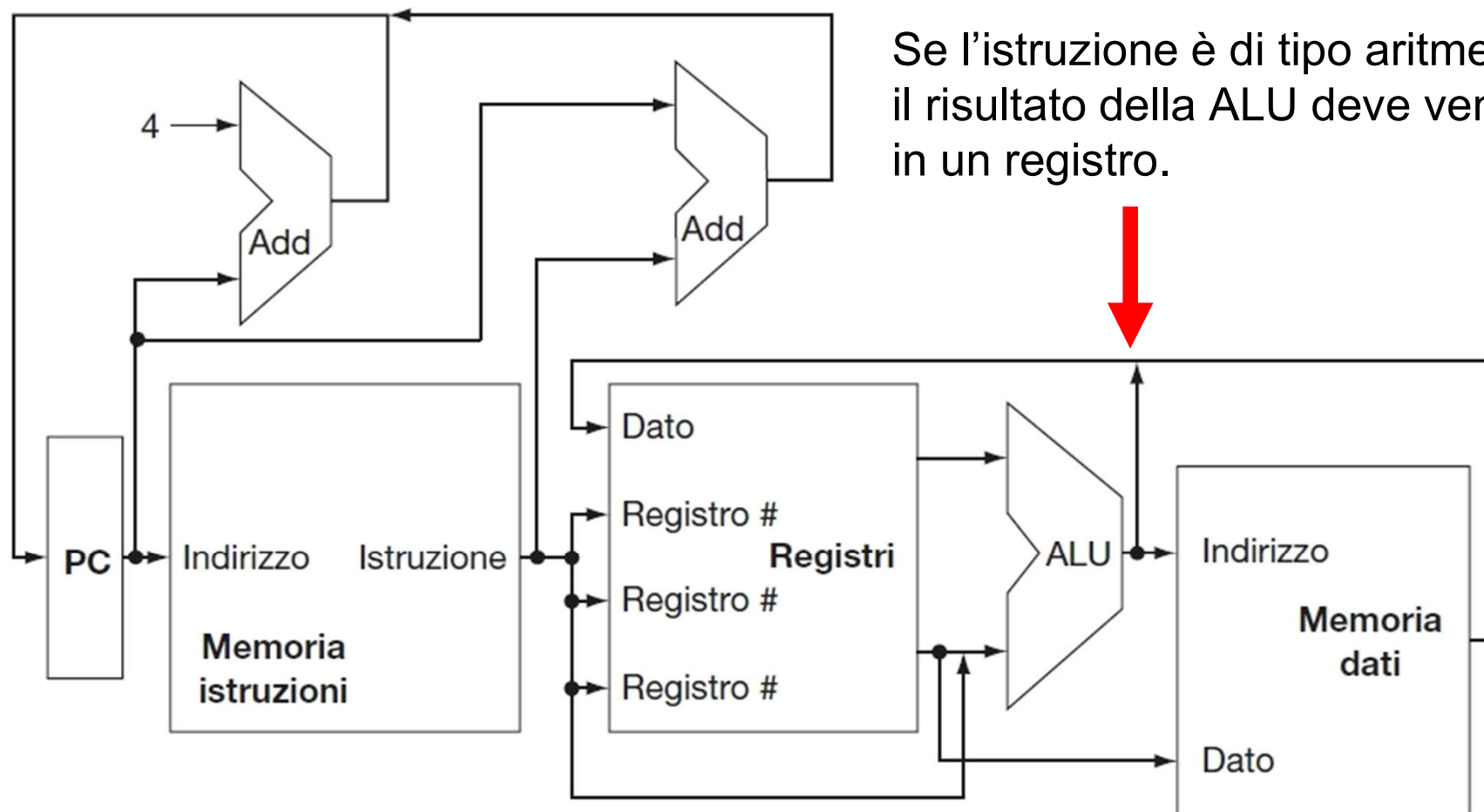
(execute)

Una volta caricati gli operandi, si può:

- eseguire un calcolo effettivo (operazioni aritmetico-logiche su interi)
- elaborare il loro contenuto per determinare un indirizzo di memoria (nel caso di load o store)
- eseguire un confronto (salto condizionato)

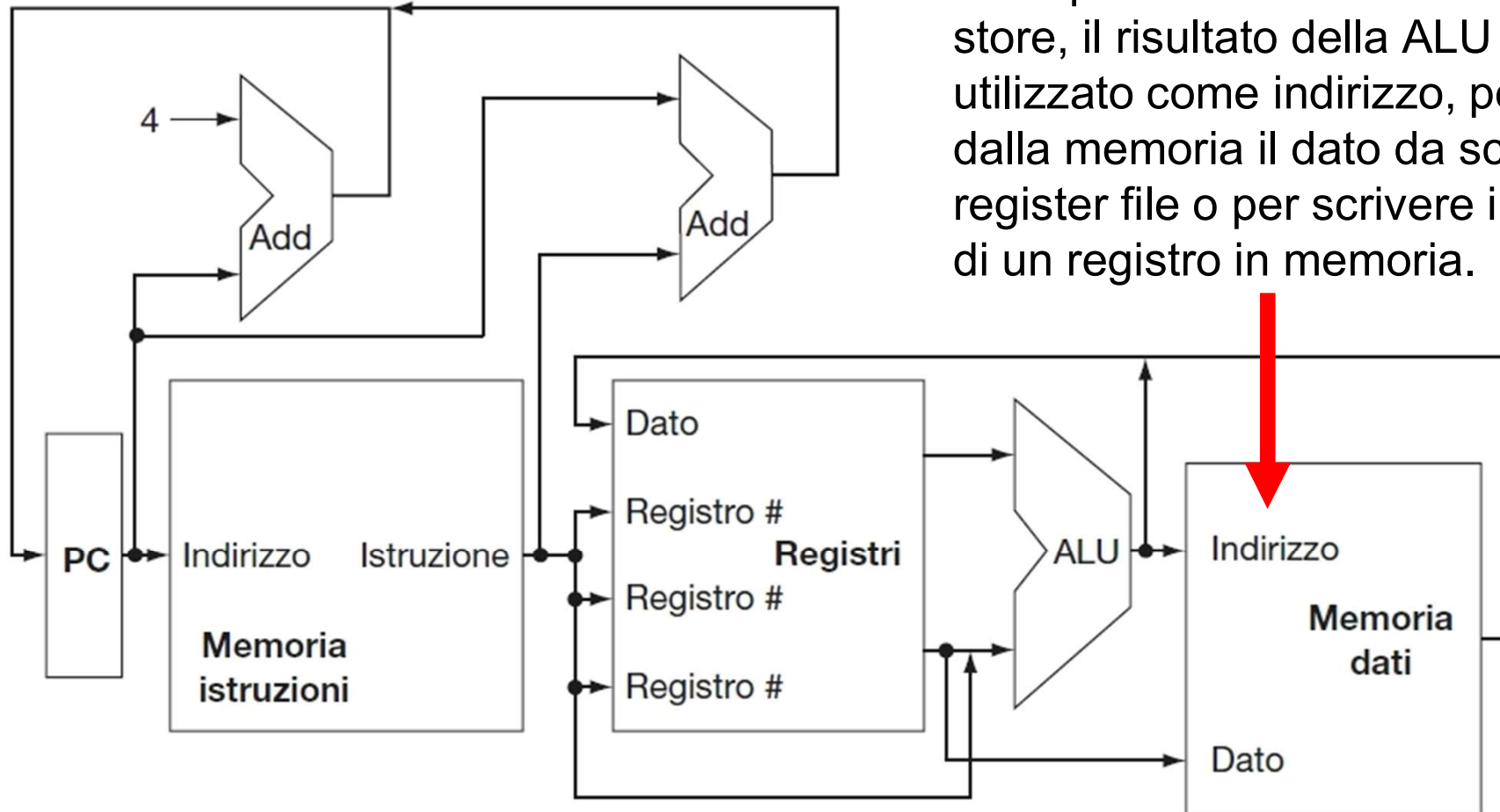


Data Path RISC-V (Semplificato)



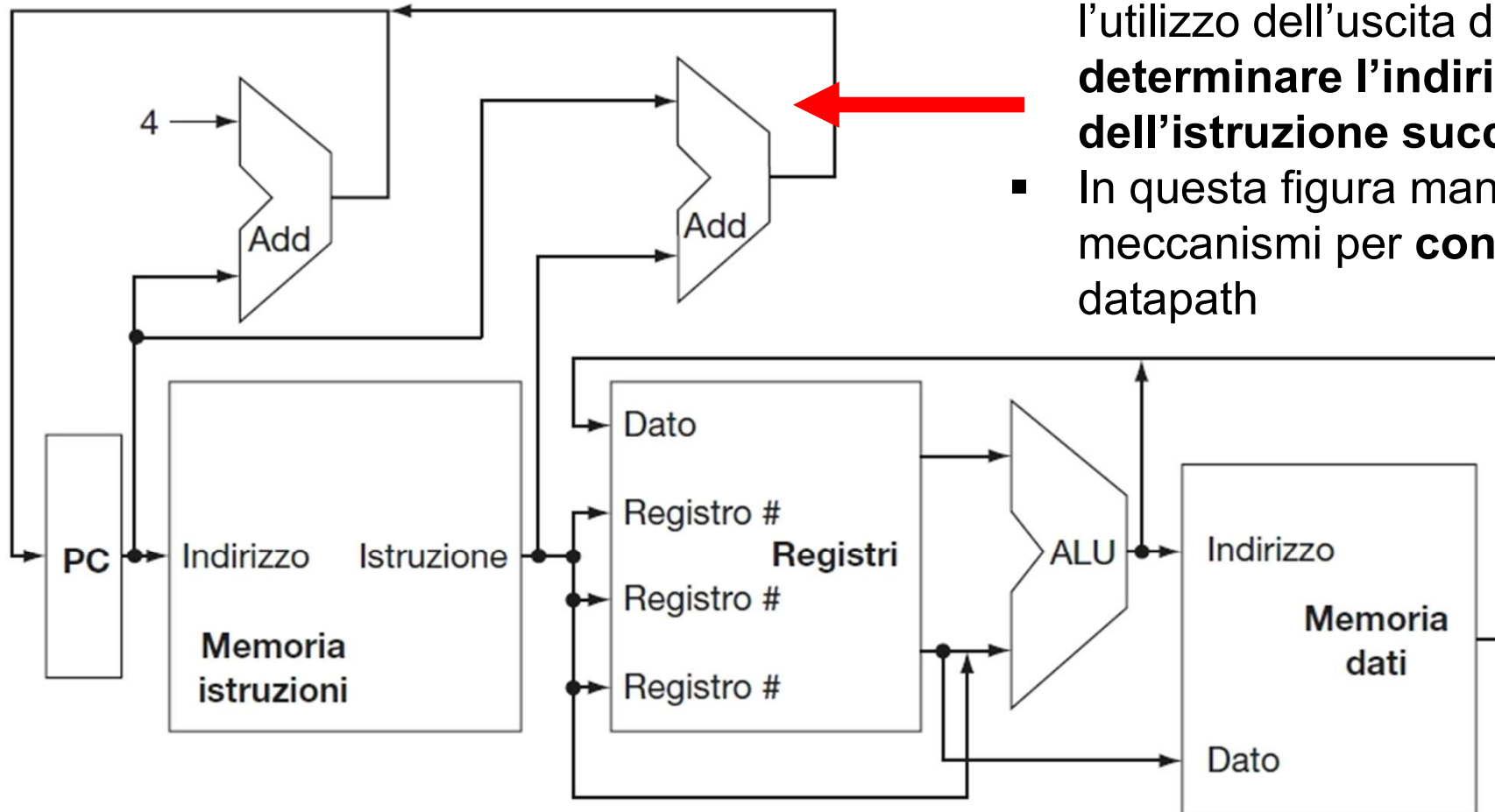
Se l'istruzione è di tipo aritmetico-logico,
il risultato della ALU deve venire scritto
in un registro.

Data Path RISC-V (Semplificato)



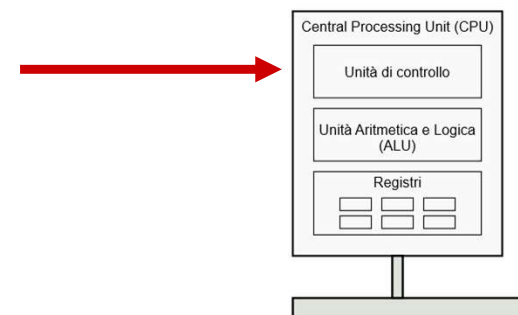
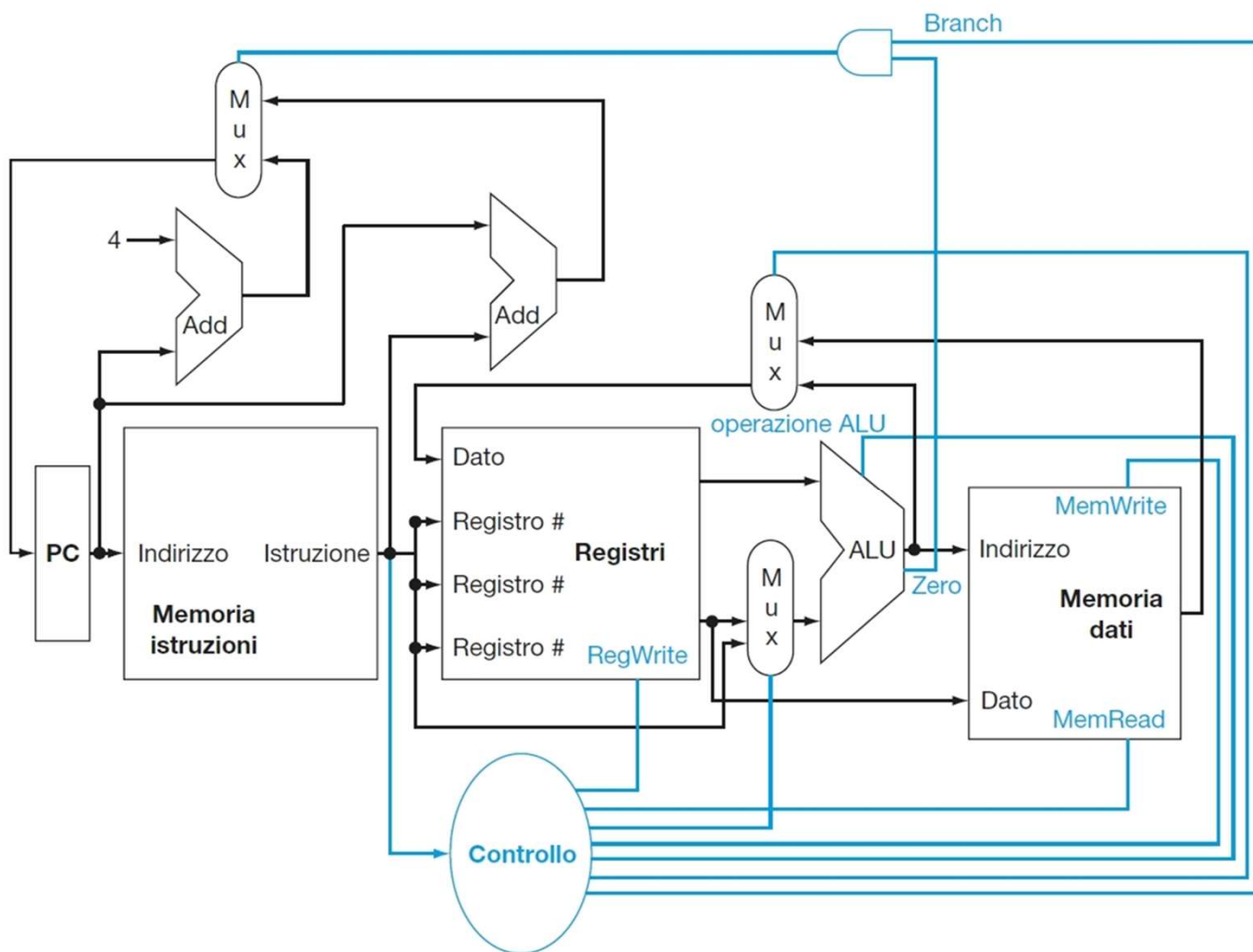
Se l'operazione è una load o una store, il risultato della ALU viene utilizzato come indirizzo, per leggere dalla memoria il dato da scrivere nel register file o per scrivere il contenuto di un registro in memoria.

Data Path RISC-V (Semplificato)



- I salti condizionati richiedono l'utilizzo dell'uscita della ALU per **determinare l'indirizzo dell'istruzione successiva**
- In questa figura mancano però i meccanismi per **controllare** il datapath

L'unità di Controllo

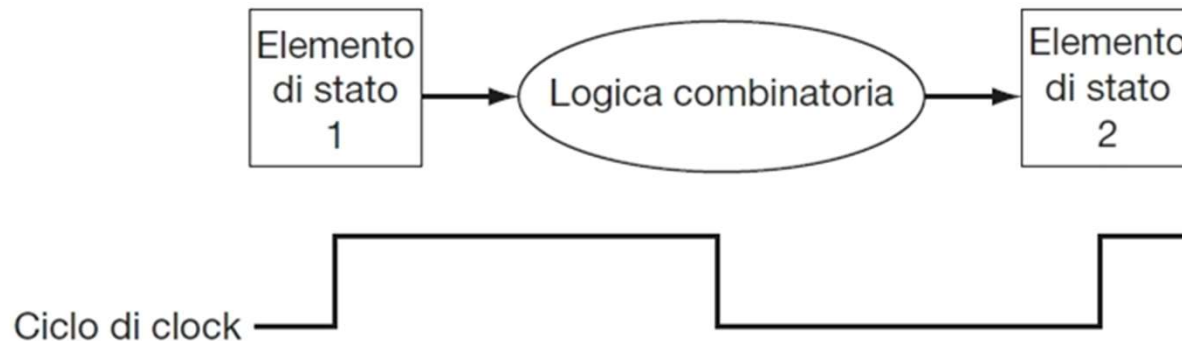


L'unità di controllo contiene i circuiti che effettivamente decodificano le istruzioni

Vedremo nel dettaglio tutti gli elementi della figura per capire l'architettura

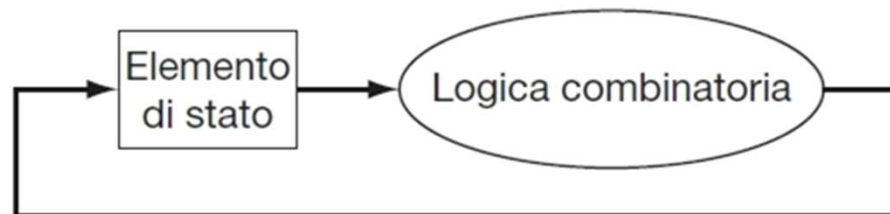
Metodologia di Temporizzazione

- l'approccio utilizzato per determinare quando un dato è valido e stabile in relazione al segnale di clock.
- **Temporizzazione sensibile ai fronti (edge-triggered)**: una tecnica di temporizzazione nella quale tutti i cambiamenti di stato avvengono su un fronte del segnale di clock
- In un sistema digitale **sincrono**, il segnale di clock determina quando gli elementi di stato scrivono la loro memoria interna



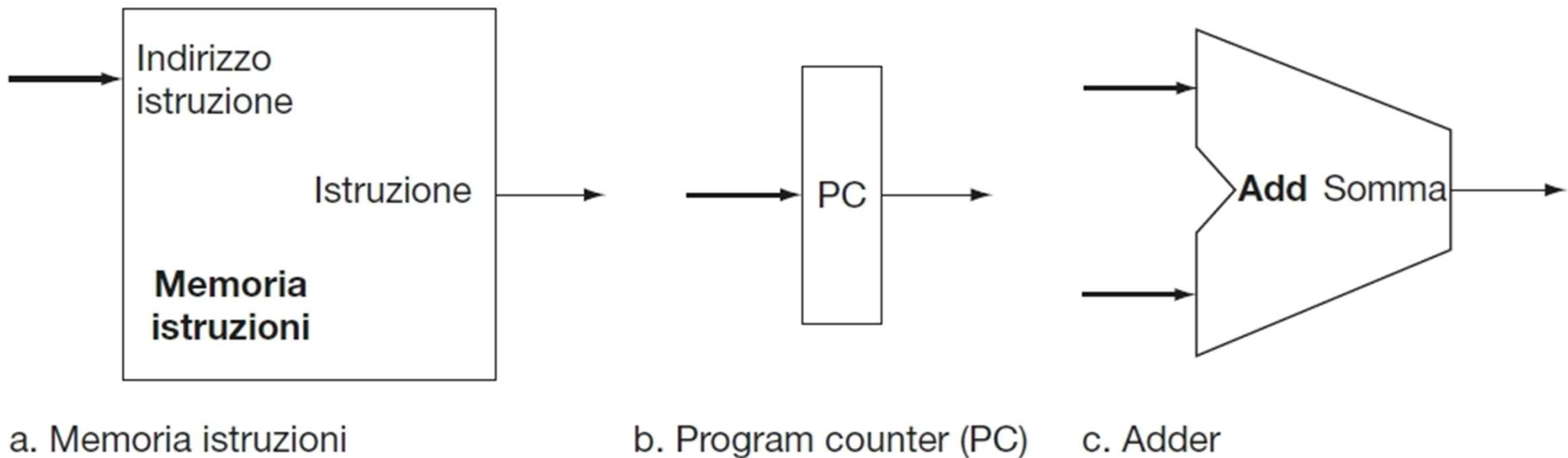
Metodologia di Temporizzazione

- Il segnale di clock determina **quando gli elementi di stato scrivono** la loro memoria interna
- Gli ingressi a un elemento di stato **devono raggiungere un valore stabile** prima che il fronte attivo del clock provochi l'aggiornamento dello stato
- Supporremo che tutti gli elementi di stato, comprese le memorie, siano sensibili al **fronte di salita**
- **La metodologia sensibile ai fronti permette di leggere e scrivere un elemento di stato nello stesso ciclo di clock**
- Il ciclo di clock deve avere una **durata sufficiente a garantire che gli ingressi siano stabili** quando arriva il fronte attivo del clock



Realizzazione di un'Unità di Elaborazione

- Prima parte: **Il Program Counter**
- Ci servono alcuni circuiti digitali tra quelli che abbiamo già visto



Memoria delle Istruzioni

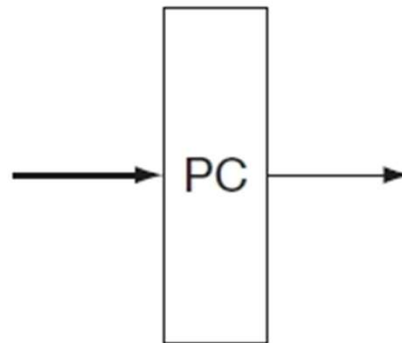


Un'unità di memoria in cui salvare le istruzioni del programma che sia in grado di fornire in uscita l'istruzione associata all'indirizzo dato in ingresso

a. Memoria istruzioni

Il Program Counter

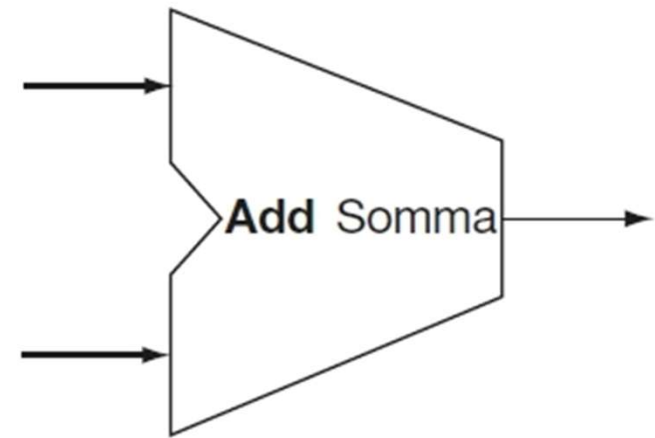
- Registro utilizzato per memorizzare l'indirizzo dell'istruzione corrente
- La memoria è **"byte addressed"**
- **Registro da 32 bit in questa architettura**



b. Program counter (PC)

Incremento del Program Counter

- Un sommatore per incrementare il PC e ottenere l'indirizzo dell'istruzione successiva
- Tutte le istruzioni RISC-V "core" sono rappresentate con 32 bit
- Quindi, nel caso più comune dobbiamo sommare 4 byte al Program Counter

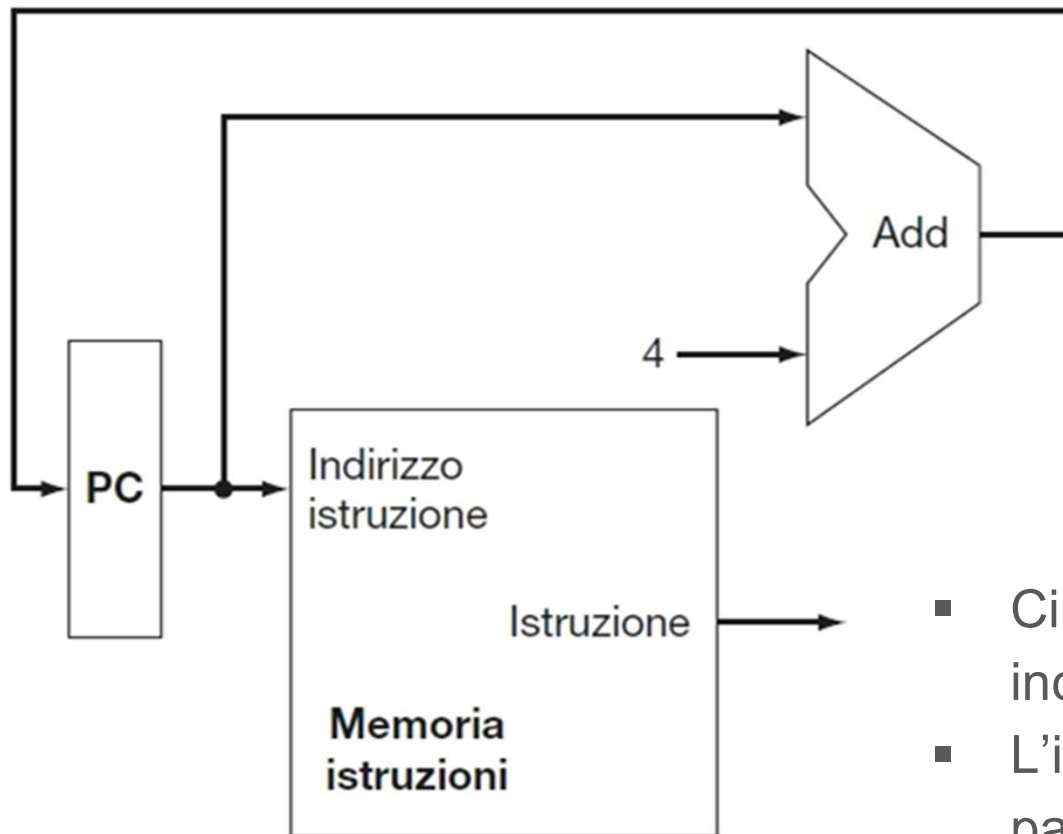


c. Adder

FORMATO DELLE ISTRUZIONI CORE

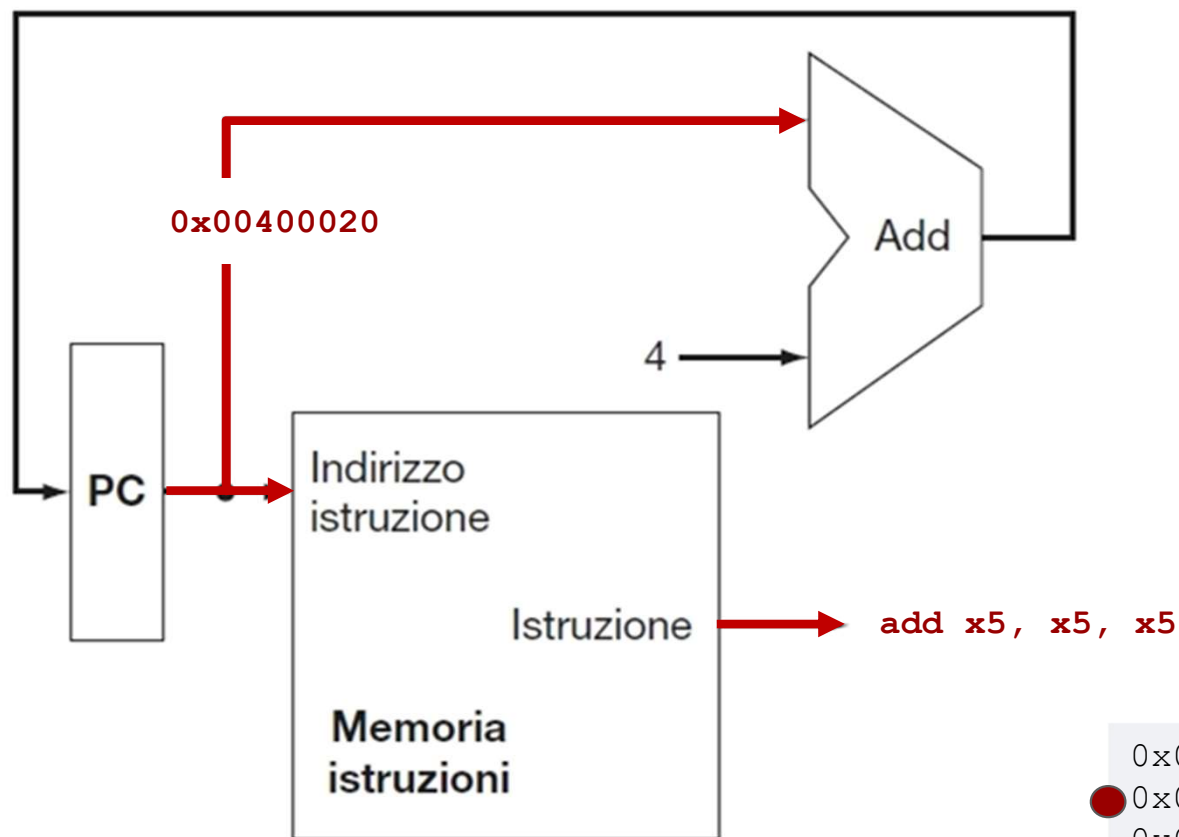
	31	27	26	25	24	20	19	15	14	12	11	7	6	0
R	funz7				rs2		rs1		funz3		rd		codop	
I	cost[11:0]						rs1		funz3		rd		codop	
S	cost[11:5]				rs2		rs1		funz3		cost[4:0]		codop	
SB	cost[12 10:5]				rs2		rs1		funz3		cost[4:1 11]		codop	
U	cost[31:12]										rd		codop	
UJ	cost[20 10:1 11 19:12]										rd		codop	

Circuito per Incrementare il Program Counter

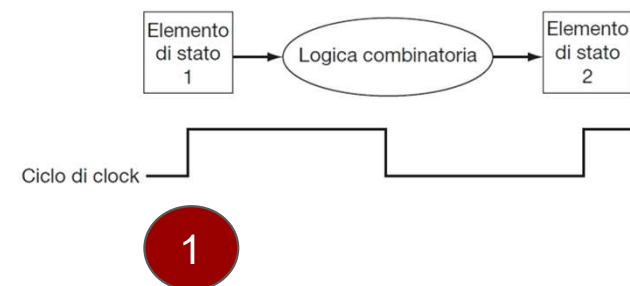


- Circuito per prelevare le istruzioni e incrementare il program counter
- L'istruzione viene utilizzata dalle altre parti dell'unità di elaborazione

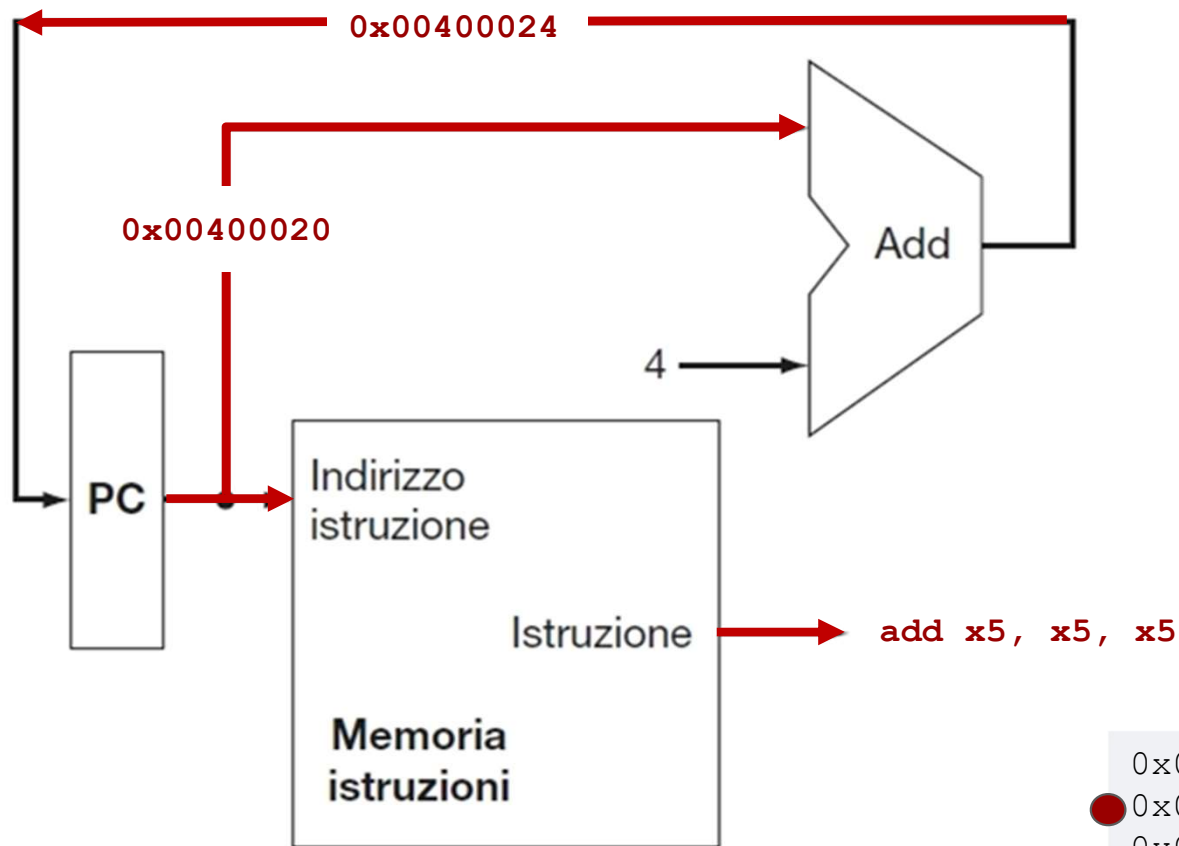
Incremento del Program Counter



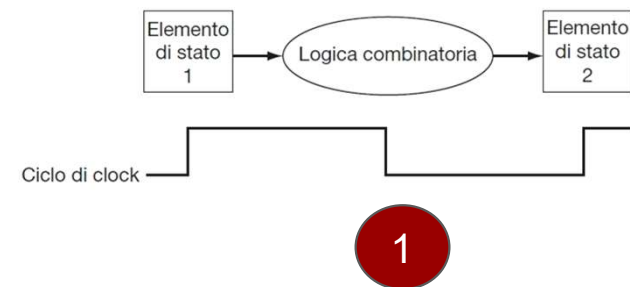
0x00400016	add x4, x4, x4
● 0x00400020	add x5, x5, x5
0x00400024	add x6, x6, x6



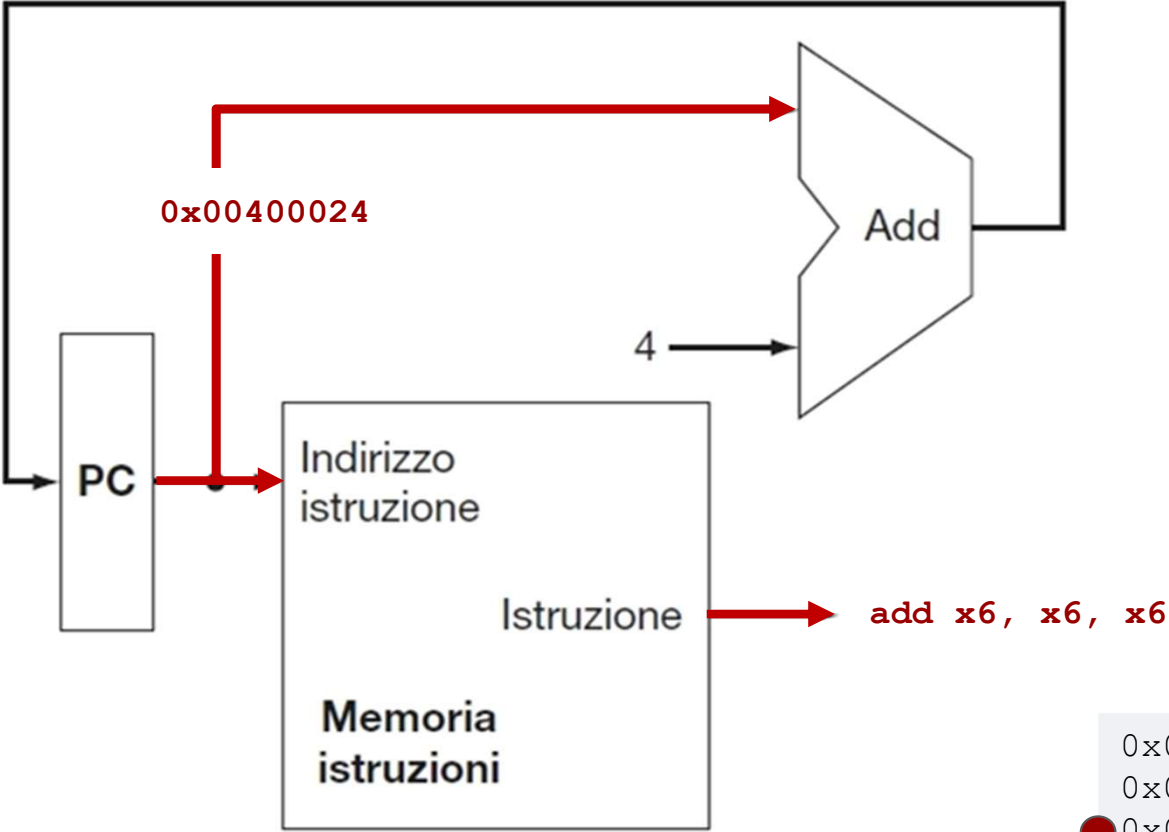
Incremento del Program Counter



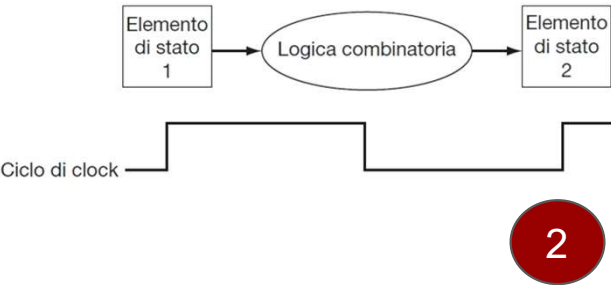
0x00400016	add x4, x4, x4
● 0x00400020	add x5, x5, x5
0x00400024	add x6, x6, x6



Incremento del Program Counter

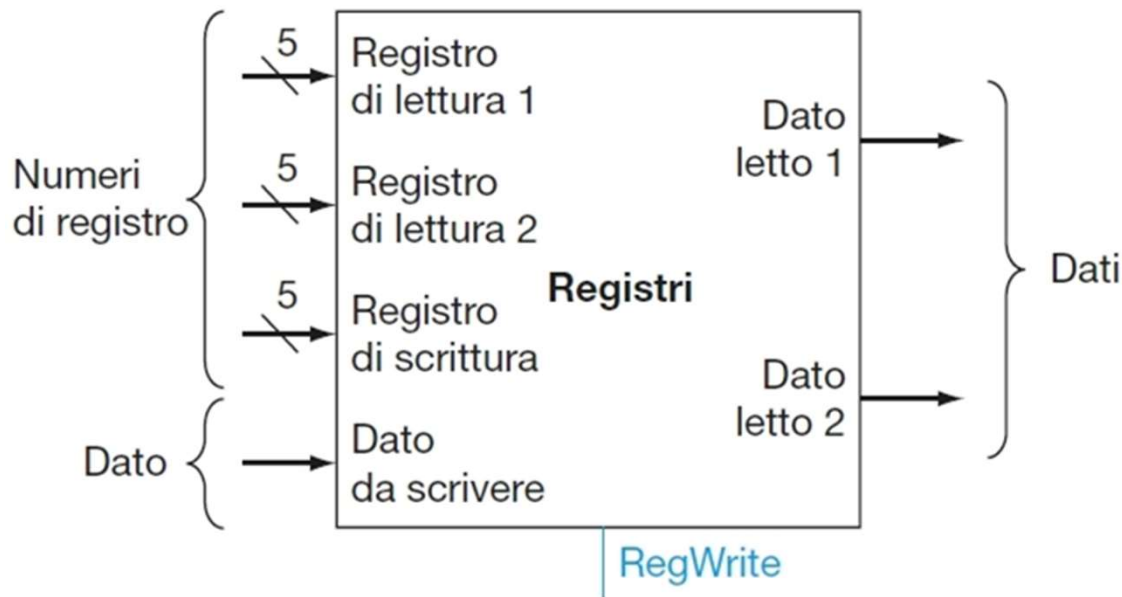


0x00400016	add x4, x4, x4
0x00400020	add x5, x5, x5
● 0x00400024	add x6, x6, x6

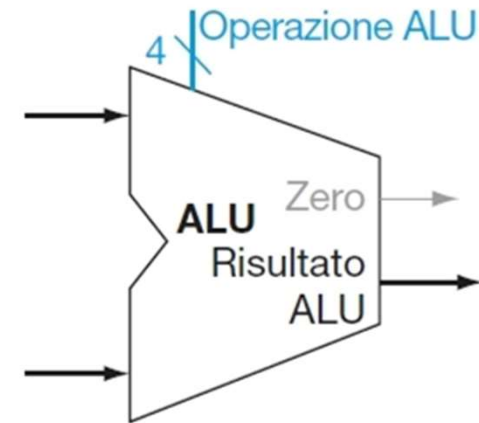


Datapath per le Istruzioni di Tipo R

- Ci servono altri due circuiti digitali (già visti)
- Register File e la ALU

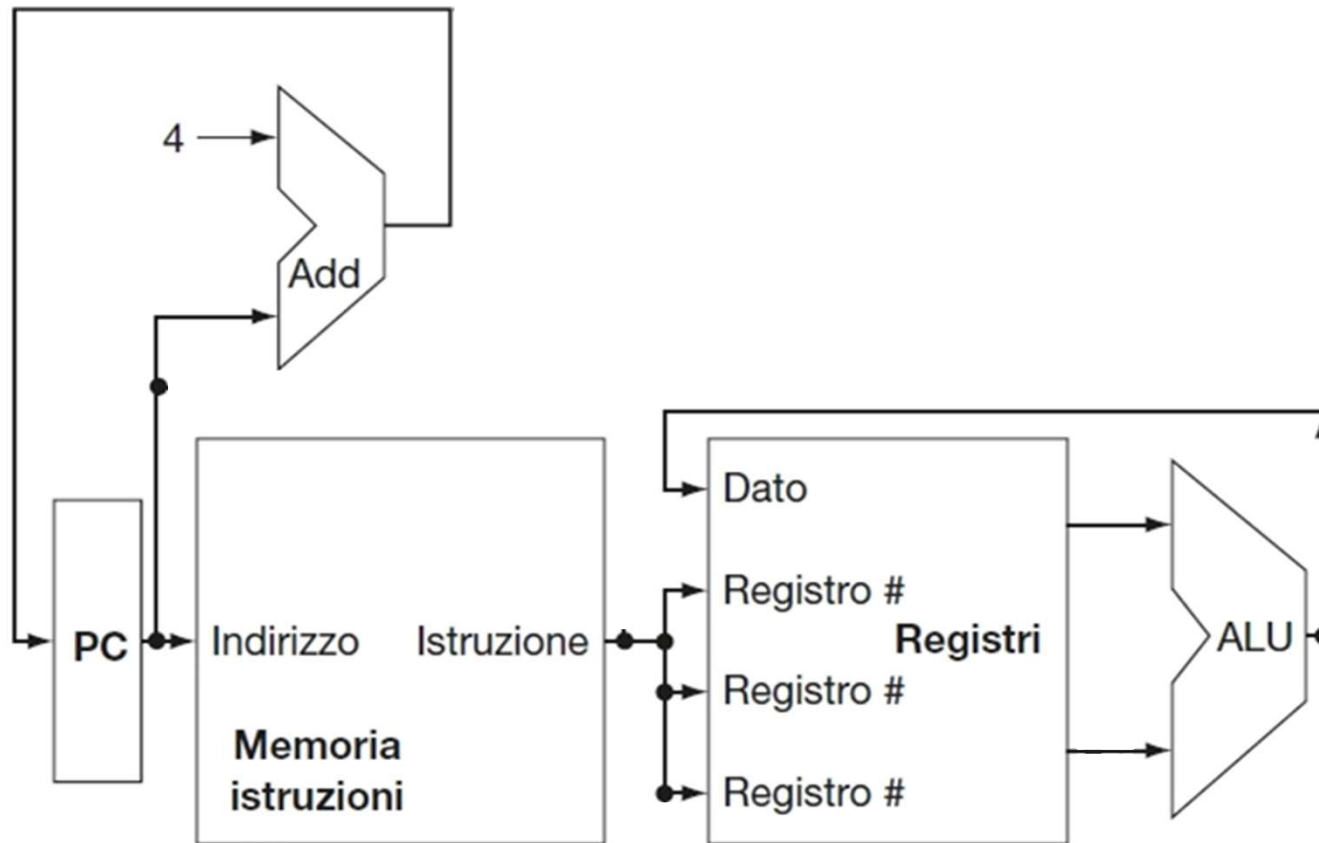


a. Registri

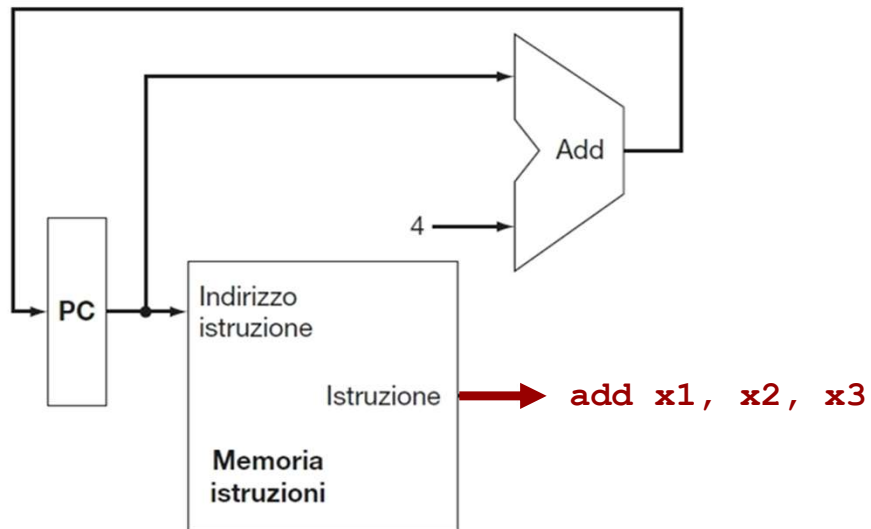


b. ALU

Datapath per le Istruzioni di Tipo R

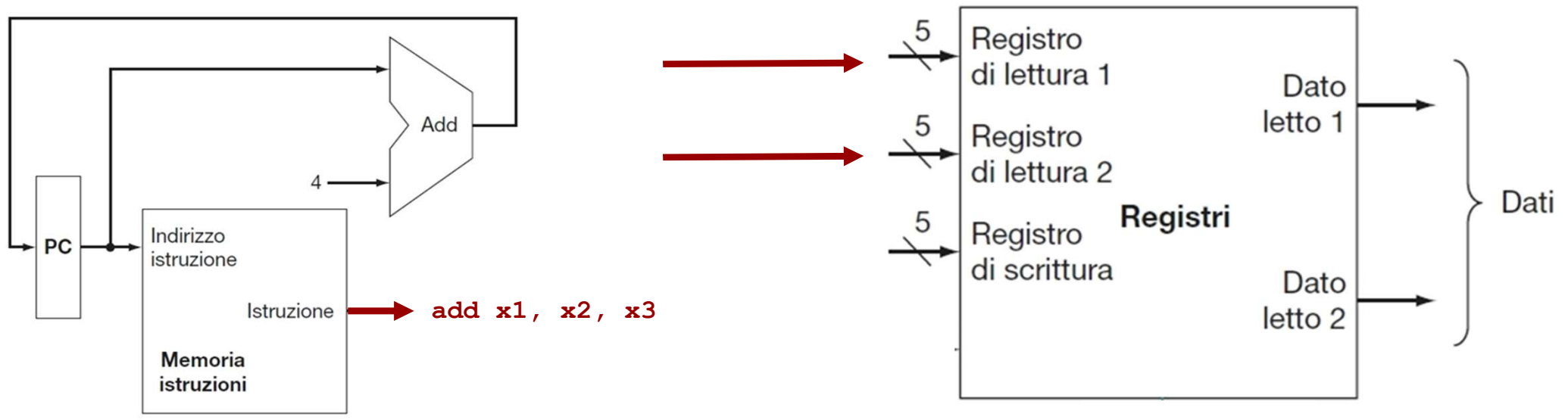


Datapath per le Istruzioni di Tipo R



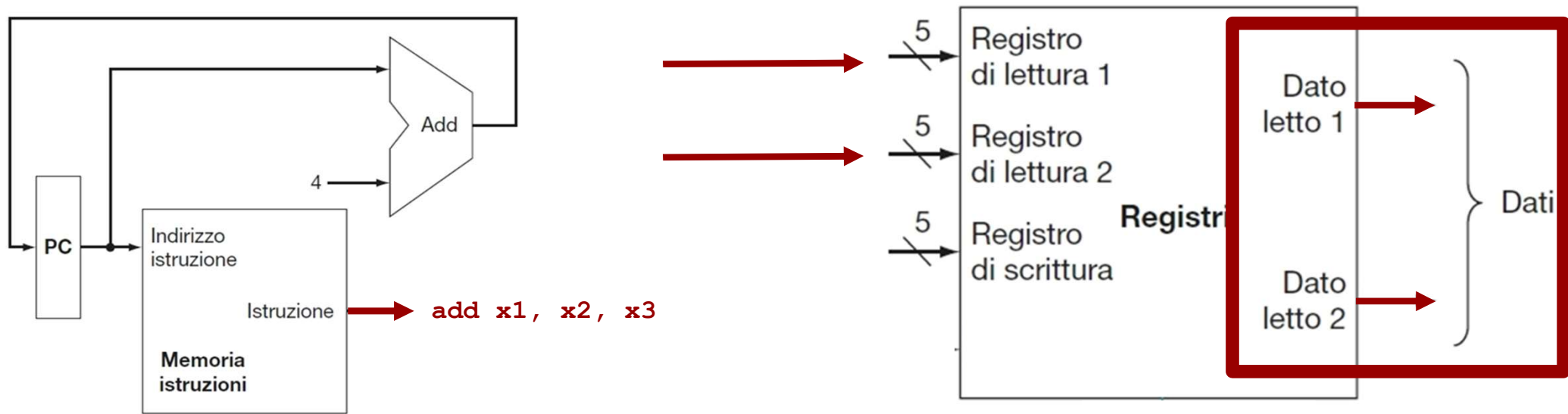
Istruzione (R)	funz7	rs2	rs1	funz3	rd	codop	Esempio
add	0000000	00011	00010	000	00001	0110011	add x1, x2, x3
sub (sottrazione)	0100000	00011	00010	000	00001	0110011	sub x1, x2, x3

Datapath per le Istruzioni di Tipo R



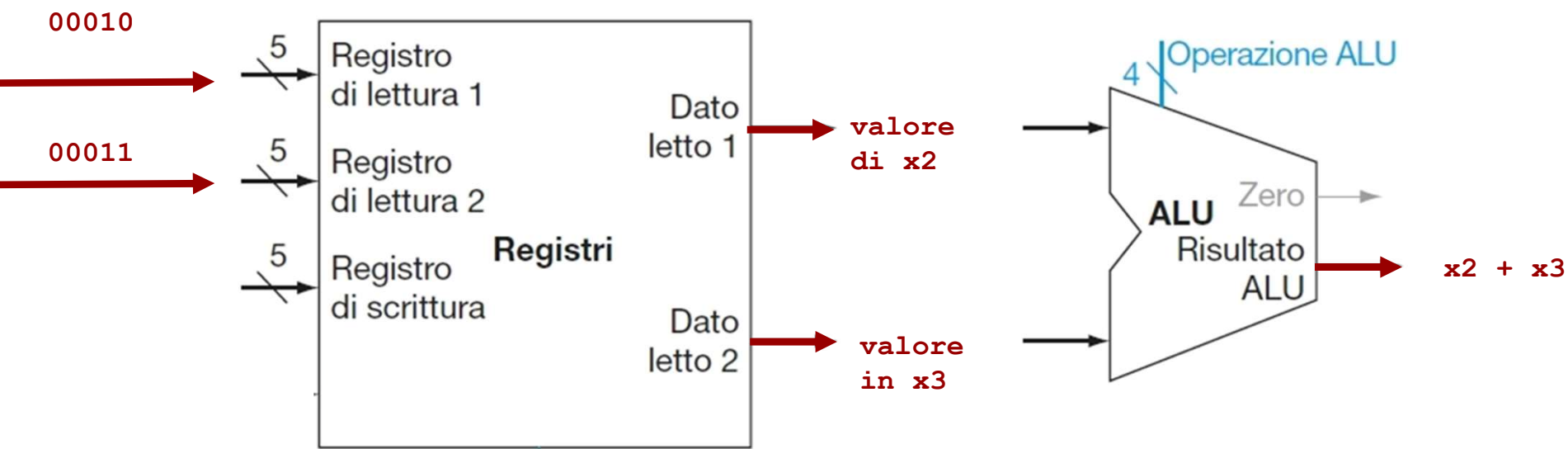
Istruzione (R)	funz7	rs2	rs1	funz3	rd	codop	Esempio
add	0000000	00011	00010	000	00001	0110011	add x1, x2, x3
sub (sottrazione)	0100000	00011	00010	000	00001	0110011	sub x1, x2, x3

Datapath per le Istruzioni di Tipo R



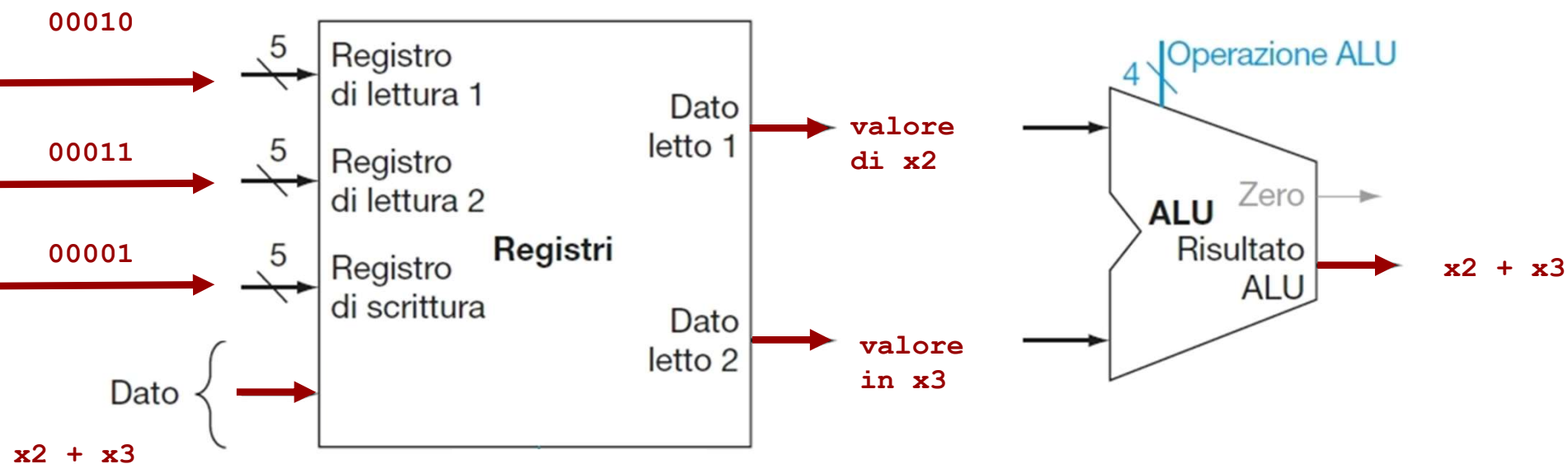
Istruzione (R)	funz7	rs2	rs1	funz3	rd	codop	Esempio
add	0000000	00011	00010	000	00001	0110011	add x1, x2, x3
sub (sottrazione)	0100000	00011	00010	000	00001	0110011	sub x1, x2, x3

Datapath per le Istruzioni di Tipo R



Istruzione (R)	funz7	rs2	rs1	funz3	rd	codop	Esempio
add	0000000	00011	00010	000	00001	0110011	add x1, x2, x3
sub (sottrazione)	0100000	00011	00010	000	00001	0110011	sub x1, x2, x3

Datapath per le Istruzioni di Tipo R



Istruzione (R)	funz7	rs2	rs1	funz3	rd	codop	Esempio
add	0000000	00011	00010	000	00001	0110011	add x1, x2, x3
sub (sottrazione)	0100000	00011	00010	000	00001	0110011	sub x1, x2, x3

Datapath per le Istruzioni di Tipo R



Per implementare le operazioni di **Tipo R** servono il **register file** e la **ALU**. Il register file contiene tutti i registri e dispone di due porte di lettura e una di scrittura.

Datapath per le Istruzioni di Tipo R



In ogni istante il register file fornisce in uscita il contenuto dei registri corrispondenti agli ingressi "Registro di lettura", senza richiedere alcun segnale di controllo.

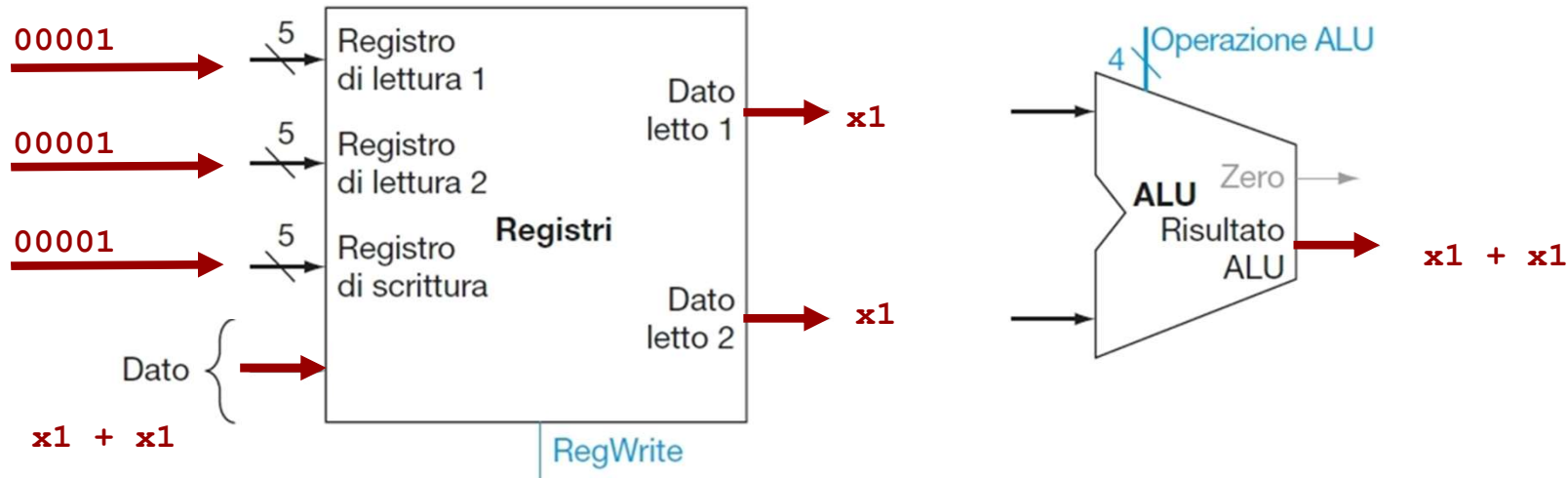
La scrittura di un registro deve essere indicata esplicitamente asserendo il segnale di controllo della scrittura, "**RegWrite**".

Datapath per le Istruzioni di Tipo R



La scrittura avviene sul **fronte attivo del clock**, per cui tutti gli ingressi interessati (cioè il dato da scrivere, il numero di registro e il segnale di controllo) devono essere validi sul fronte attivo del clock.

Datapath per le Istruzioni di Tipo R



Essendo la scrittura attiva sui fronti, l'architettura può leggere e scrivere lo stesso registro nello stesso ciclo di clock (e.g., **add x1, x1, x1** è lecita).

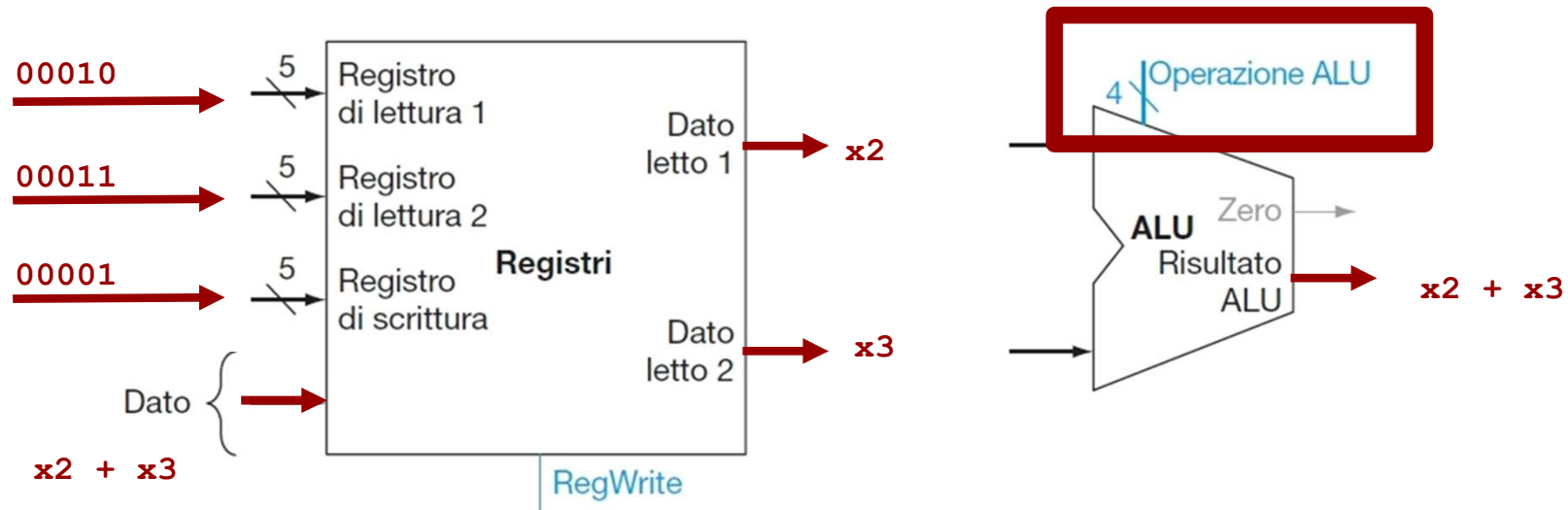
La **lettura** fornirà il **dato** contenuto nel registro **scritto in un ciclo di clock precedente**, mentre il valore scritto potrà essere letto nel ciclo di clock successivo.

Datapath per le Istruzioni di Tipo R



Gli ingressi che specificano al register file il numero d'ordine dei registri hanno ampiezza di **5 bit**, mentre i **bus che trasportano i dati sono ampi 32 bit**.

Datapath per le Istruzioni di Tipo R

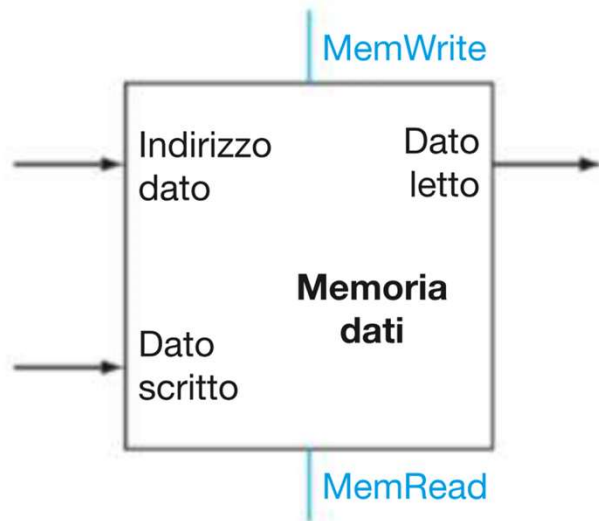


L'operazione che la ALU deve eseguire viene specificata dal segnale **"Operazione ALU"**, che è ampio 4 bit. Il circuito di controllo della ALU sarà visto nel dettaglio dopo.

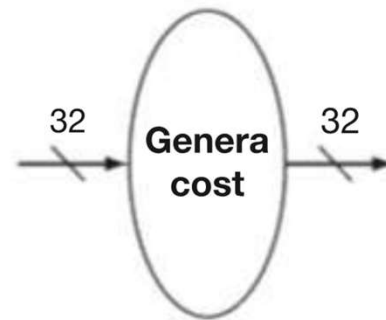
L'uscita "Zero" della ALU, verrà utilizzata per implementare i salti condizionati.

Istruzioni Load (Tipo I) e Store (Tipo S)

- Ci servono ancora altri due circuiti digitali
- La **Memoria Dati** e un'**Unità di Generazione della Costante** (nel libro in Italiano chiamata anche di "Unità di estensione del segno")



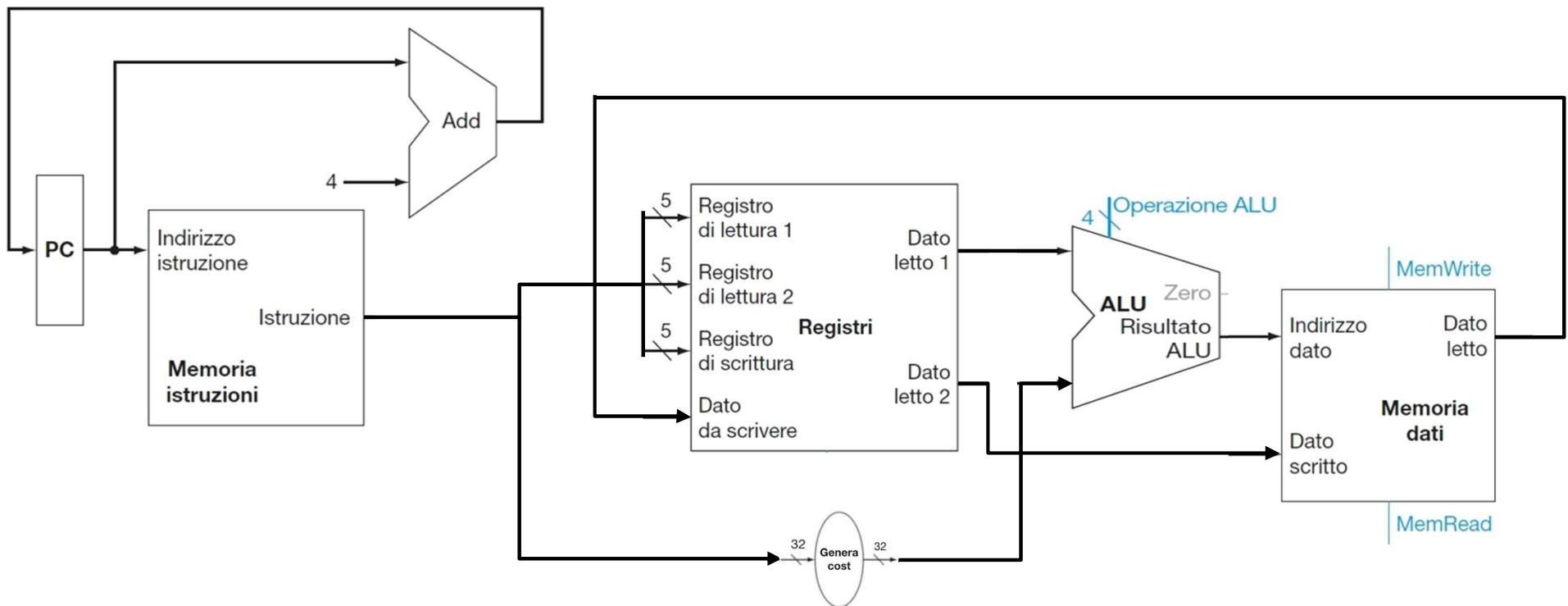
(a) Unità di memoria dati



(b) Unità di estensione del segno

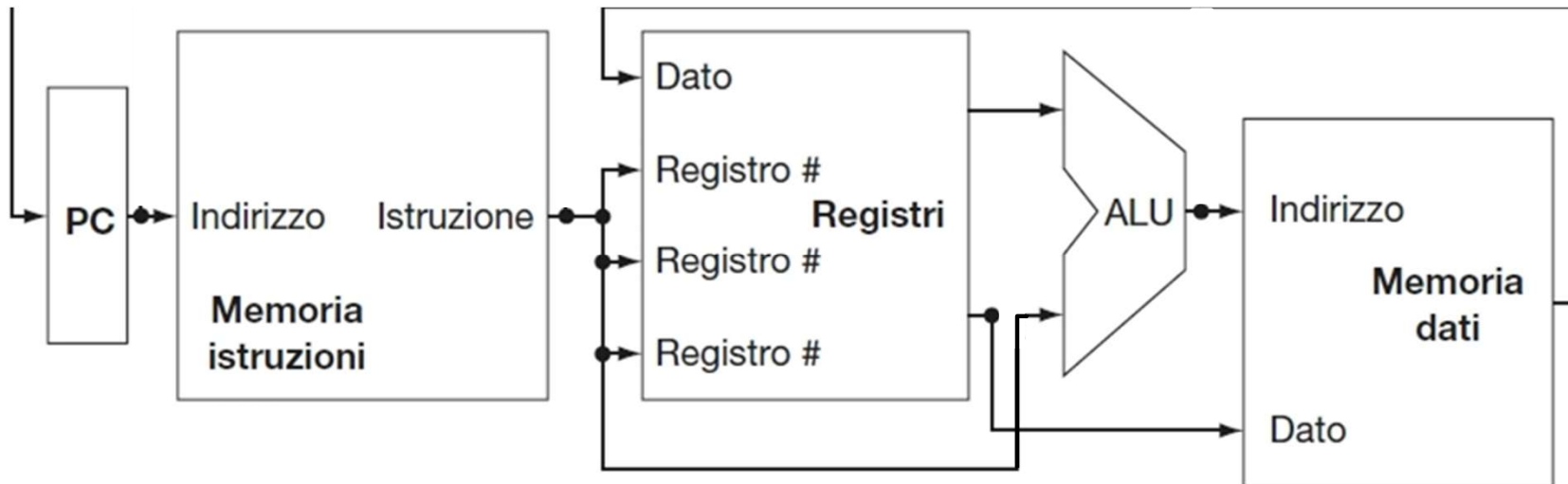
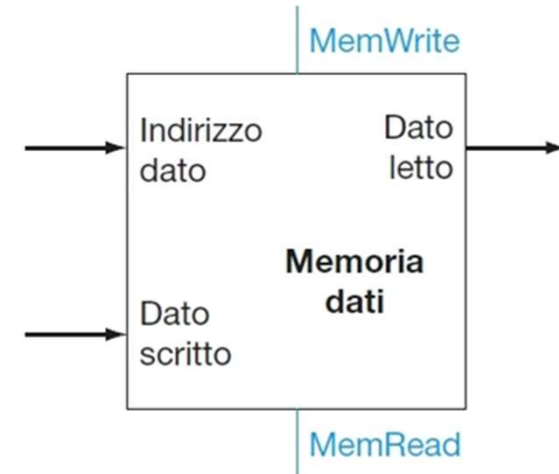
Istruzioni Load (Tipo I) e Store (Tipo S)

I	cost[11:0]		rs1	funz3	rd	codop
S	cost[11:5]	rs2	rs1	funz3	cost[4:0]	codop



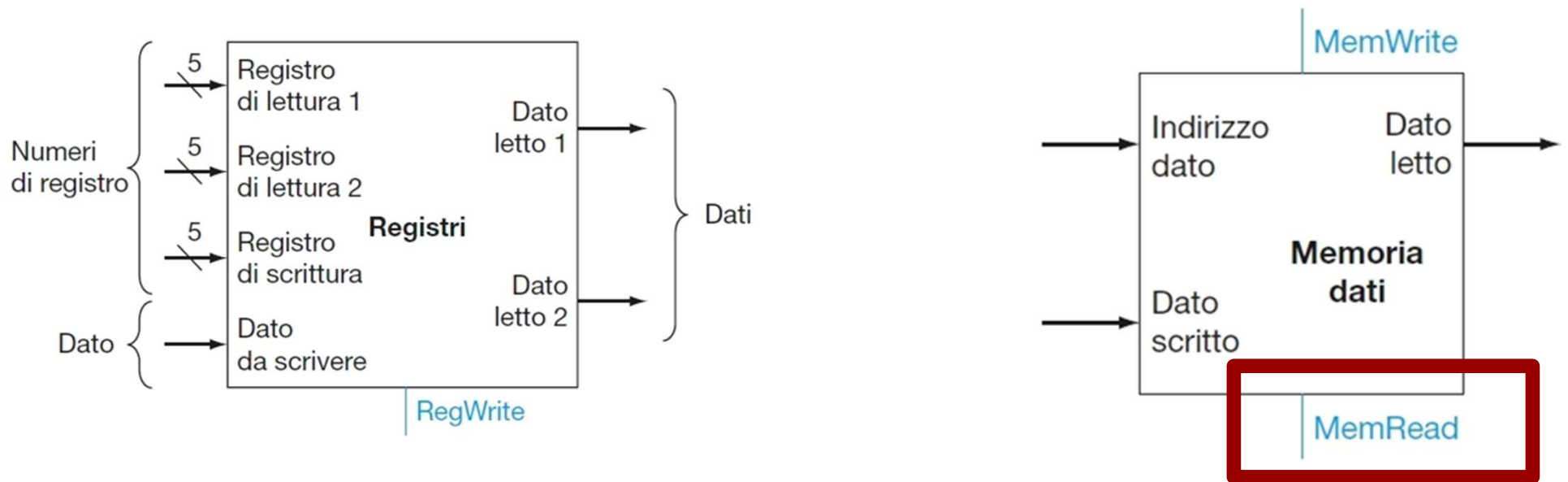
Istruzioni Load (Tipo I) e Store (Tipo S)

- L'unità di memoria è un elemento di stato
- Ingressi
 - l'indirizzo del dato (della ALU) **lw x1, offset(x2)**
 - Il dato da scrivere (dai Registri) **sw x1, offset(x2)**
- Uscita:
 - il dato letto (ai Registri)



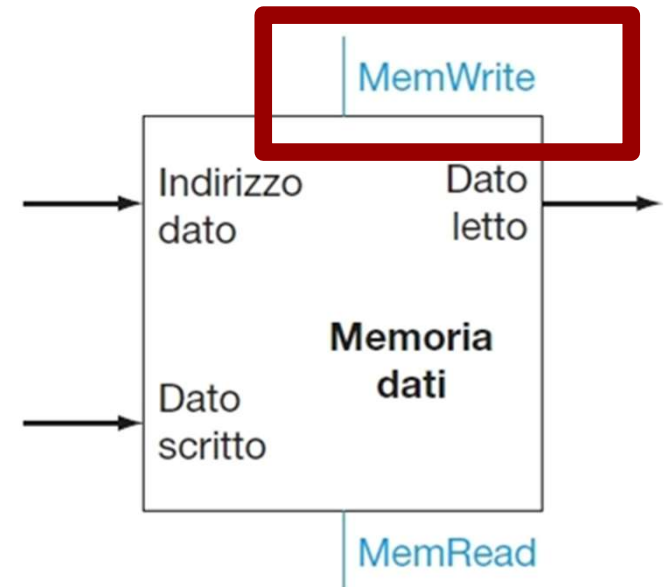
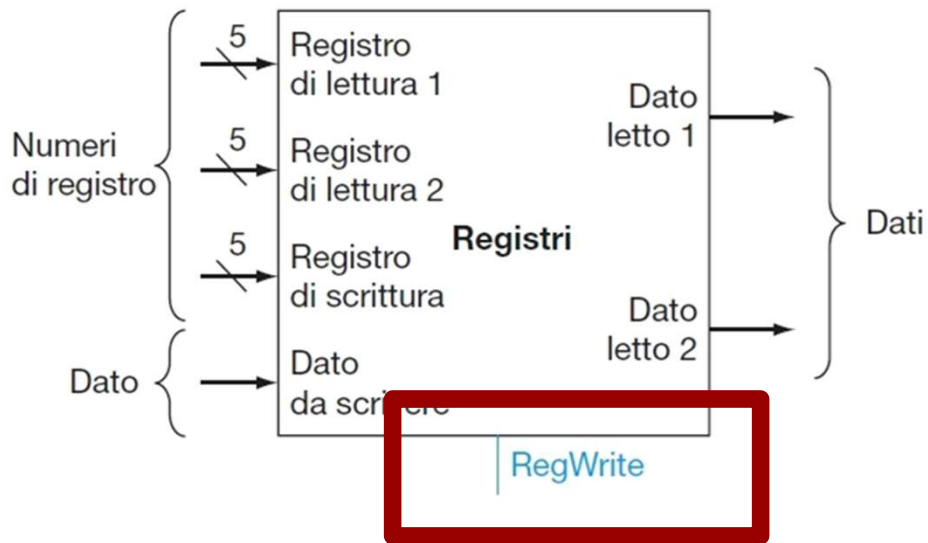
Istruzioni Load (Tipo I) e Store (Tipo S)

- L'unità di memoria ha bisogno di un segnale di lettura esplicito, diversamente da quanto avviene per il register file.



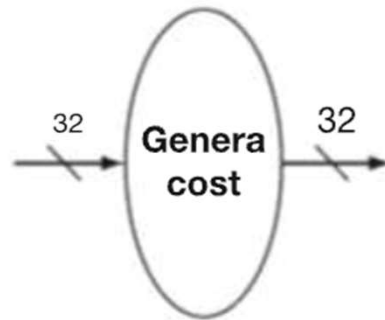
Istruzioni Load (Tipo I) e Store (Tipo S)

- Come nel caso del funzionamento dei Registri, (si suppone che) la scrittura della memoria è attiva su un fronte del segnale di clock



Istruzioni Load (Tipo I) e Store (Tipo S)

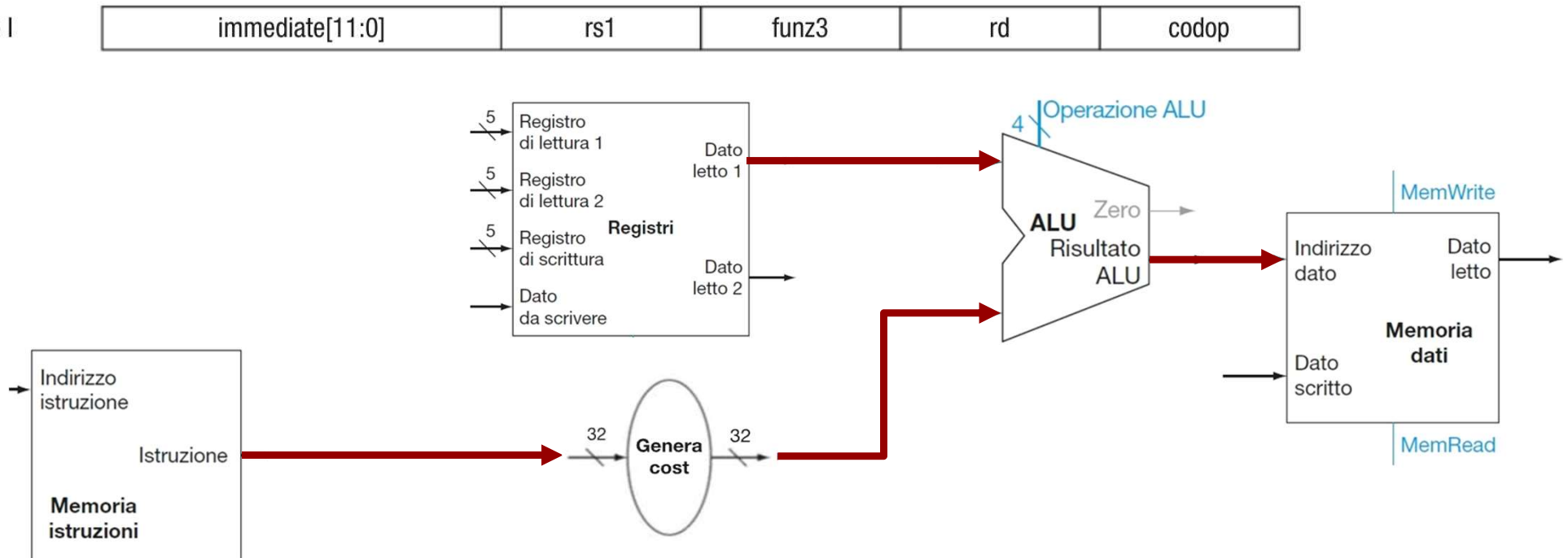
- Generazione della Costante: è necessario selezionare tra i 32 bit dell'istruzione i 12 bit che compongono il campo immediato.
- Dipende dal tipo (e formato) dell'istruzione.
- Estensione del segno: aumento della dimensione di un dato ottenuta replicando il bit più significativo, che rappresenta il segno del dato originale, nei bit più significativi del dato di destinazione.



Istruzioni Load (Tipo I) e Store (Tipo S)

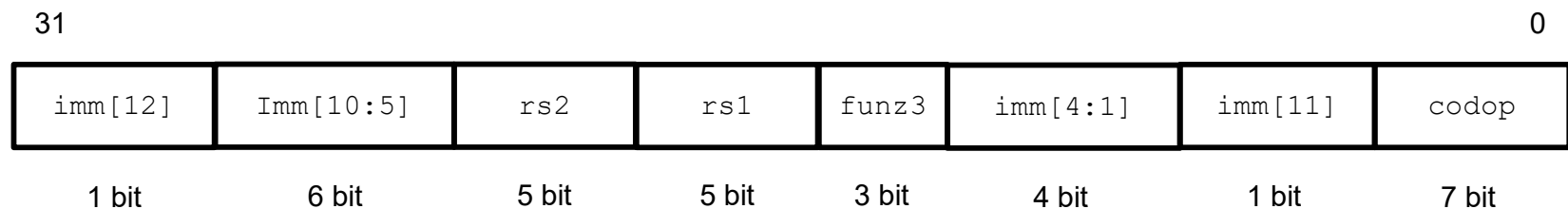
- L'unità di generazione della costante riceve in ingresso un numero a 32 bit per una load, una store e un salto condizionato all'uguaglianza
- Fornisce in uscita un numero a 32 bit dotato di segno, partendendo dal campo immediato a 12 bit.

(b) Tipo I



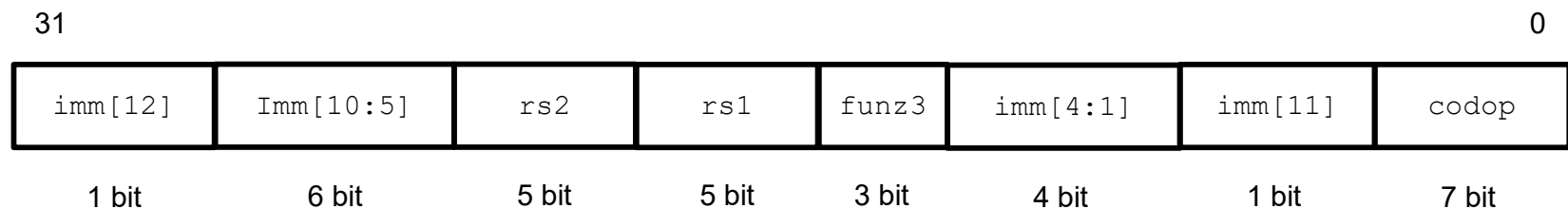
L'Istruzione di Salto **beq** (Tipo SB)

- Istruzione che richiede il confronto fra due valori e che consente il trasferimento del controllo a un altro indirizzo del programma a seconda del risultato del confronto
- Le istruzioni di salto condizionato utilizzano il **formato di tipo SB**
- Indirizzamento relativo al Program Counter (PC-relative addressing).
- Può rappresentare indirizzi di salto da -4096 a 4094, in multipli di due



L'Istruzione di Salto **beq** (Tipo SB)

- L'architettura stabilisce che il campo offset sia **spostato di 1 bit a sinistra**, in modo tale che l'offset non codifichi lo spiazzamento in numero di byte, ma in **numero di mezze parole**.
- Di fatto, tale spostamento aumenta lo spazio di indirizzamento dell'offset di un fattore 2 rispetto alla codifica dello spiazzamento in byte.
- Quindi, i 12 bit possono rappresentare indirizzi di salto da -4096 a 4094, in multipli di due



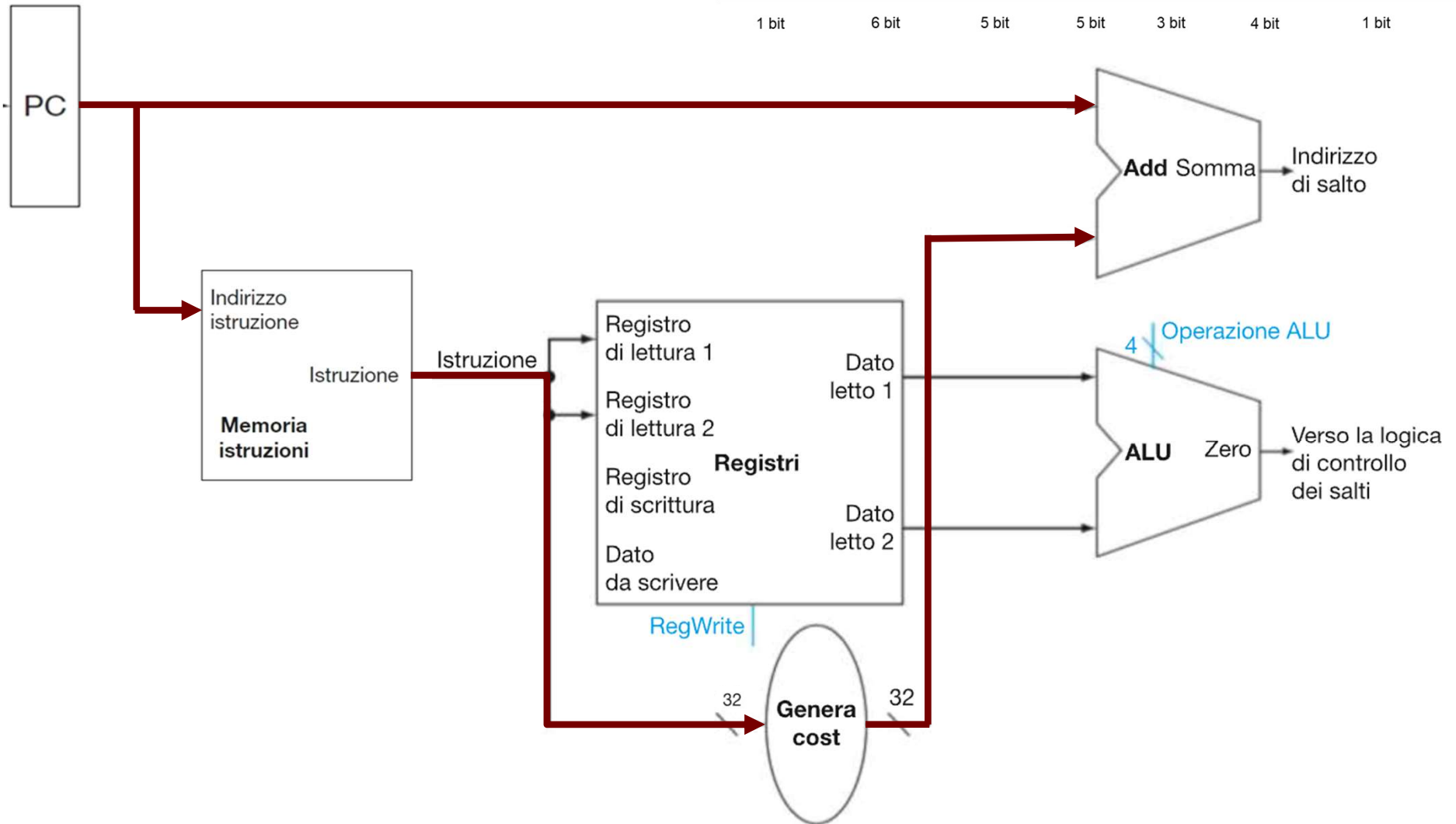
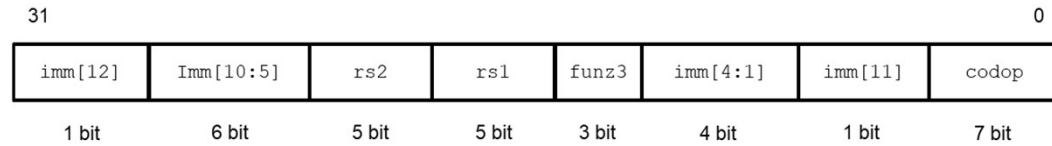
L'Istruzione di Salto **beq** (Tipo SB)

- **Indirizzo di destinazione del salto:** l'indirizzo che viene specificato in un salto condizionato
- Diventerà il nuovo indirizzo contenuto nel Program Counter (PC) se il salto condizionato viene eseguito
- L'indirizzo di destinazione di un salto è ottenuto mediante **la somma del campo offset contenuto nell'istruzione stessa con l'indirizzo dell'istruzione di salto condizionato (e.g., 0x00400020)**

0x00400020 **beq rs1, rs2, L1**

- il campo offset dell'istruzione va esteso a 32 bit con segno per la somma

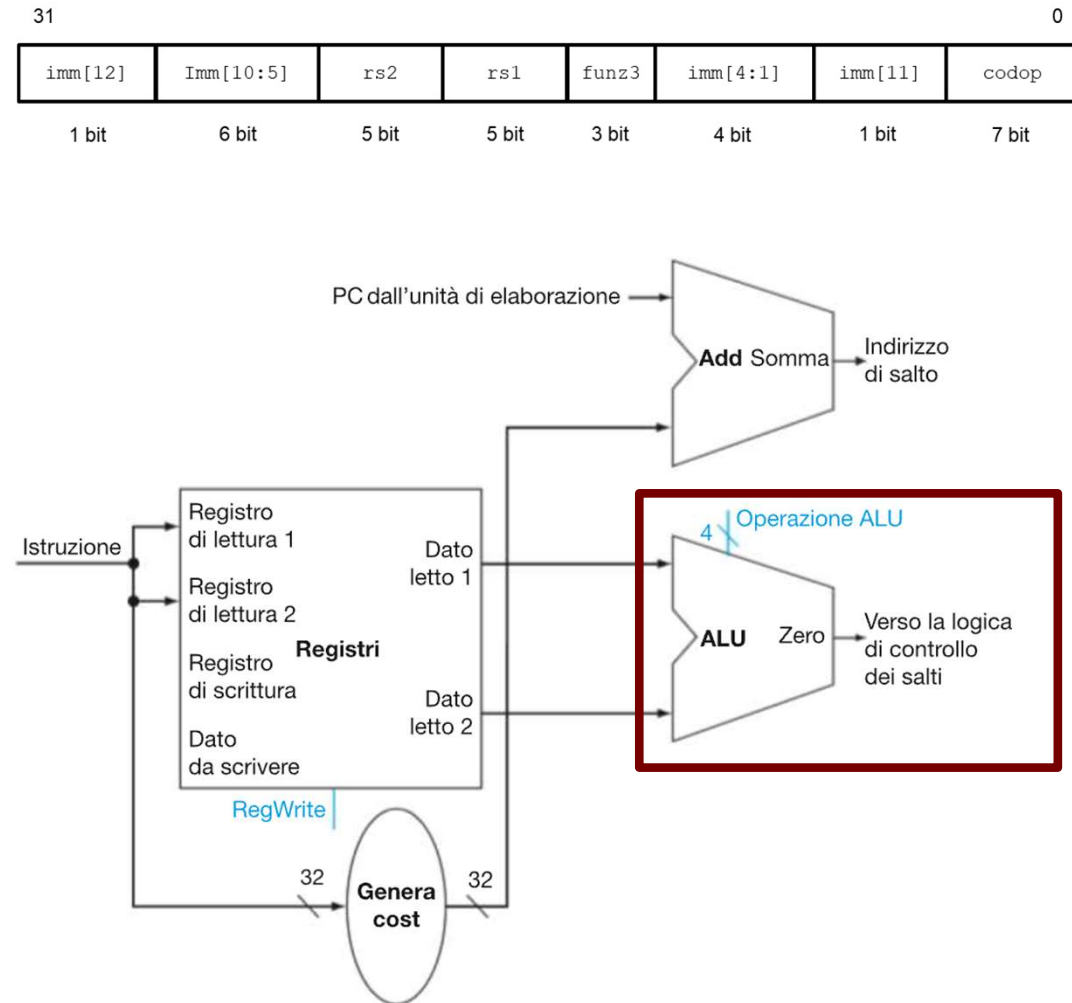
L'Istruzione di Salto **beq** (Tipo SB)



L'Istruzione di Salto **beq**

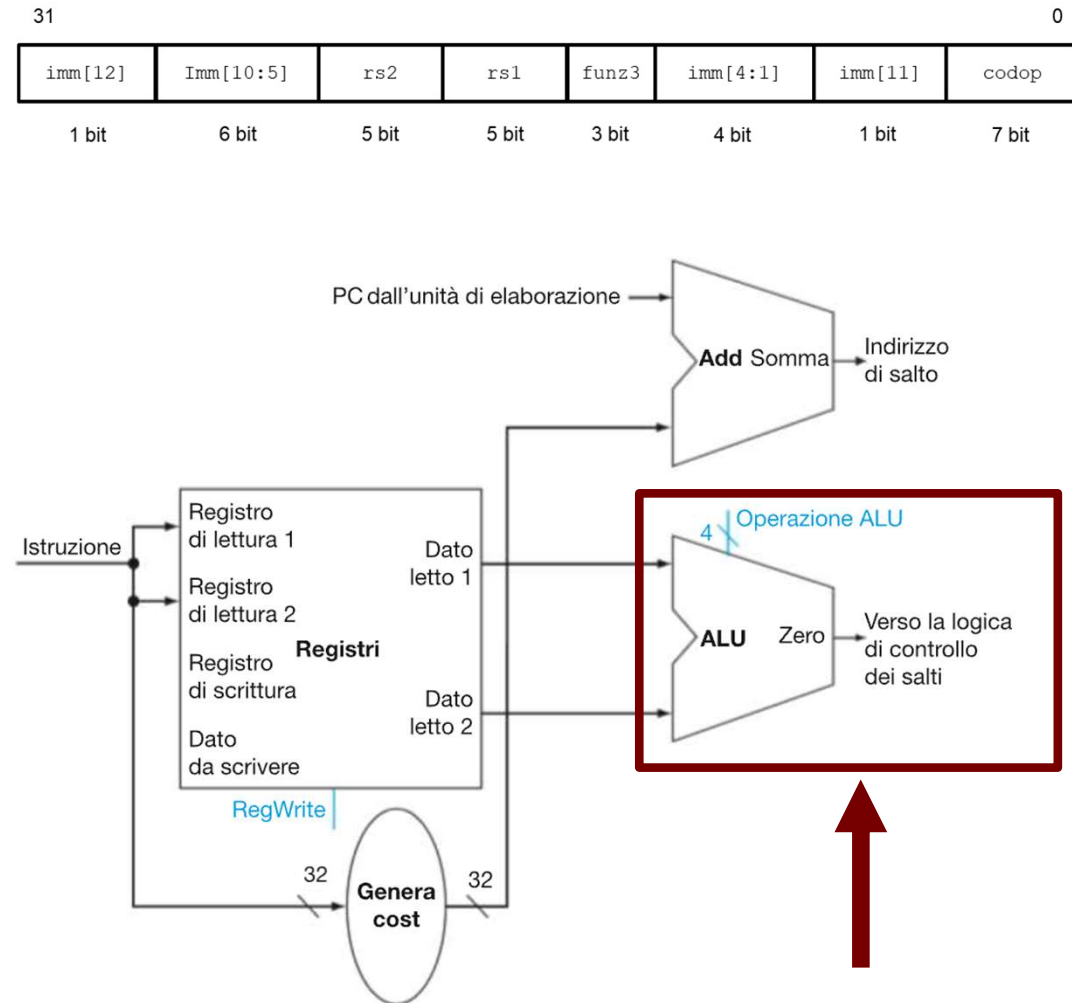
Bisogna determinare se l'istruzione da eseguire dopo sarà:

- **branch not taken, salto non preso:** presente nella posizione successiva della memoria delle istruzioni
- **branch taken, salto preso:** contenuta all'indirizzo di destinazione del salto
- **Salto incondizionato:** assimilate a dei salti condizionati che vengono sempre eseguiti



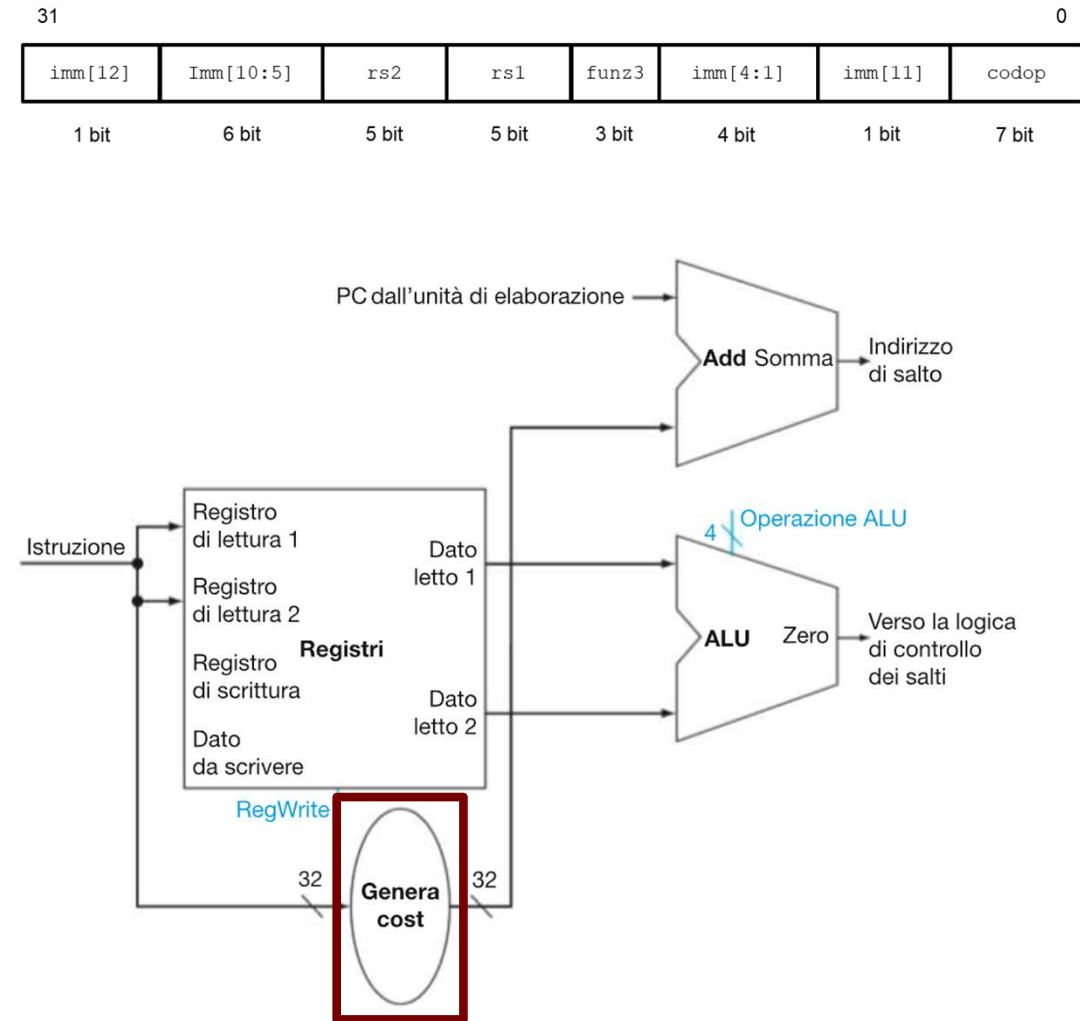
L'Istruzione di Salto **beq**

- Si legge il contenuto dei due registri contenenti gli operandi
- Il confronto tra i due operandi viene fatto dalla ALU
- La ALU fornisce un segnale in uscita che indica se il risultato è pari a 0
- Insieme ai operandi si invia alla ALU i segnali di controllo per effettuare una sottrazione
- Se l'uscita Zero della ALU viene asserita, i due operandi sono uguali



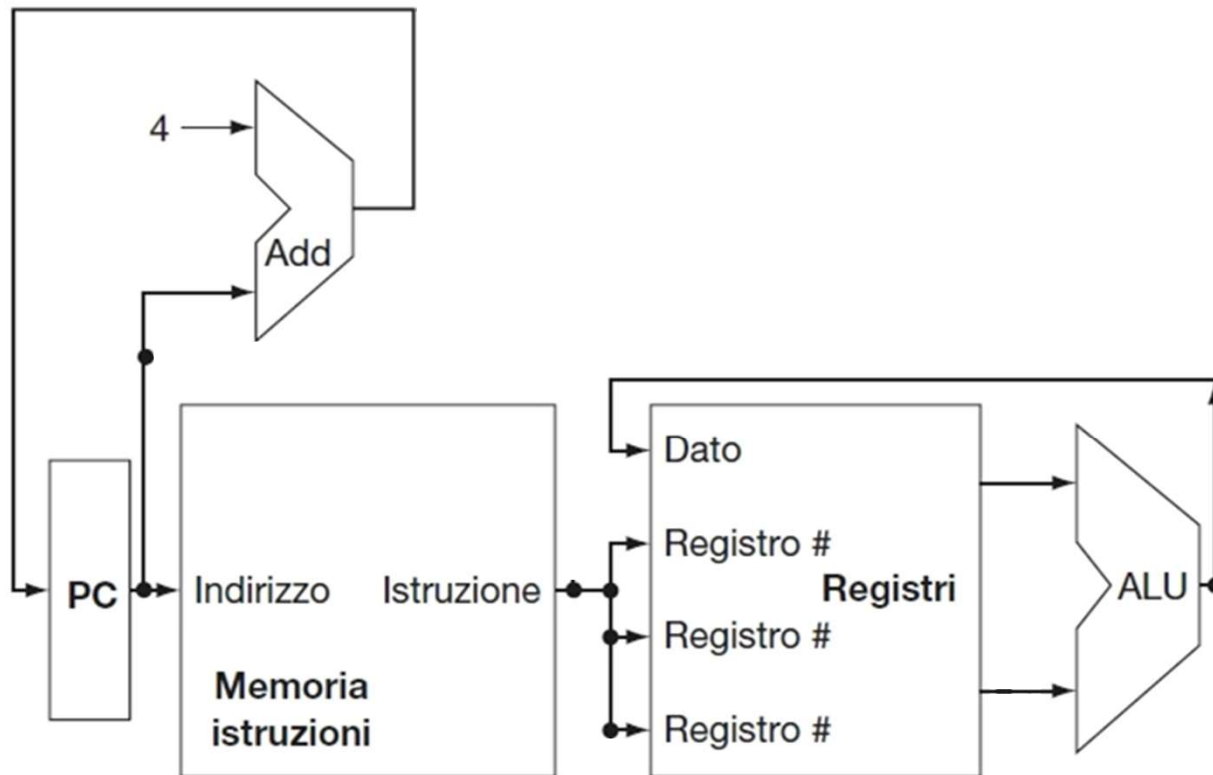
L'Istruzione di Salto **beq**

- L'unità di generazione della costante
 - prende in input i 12 bit del campo immediato
 - Effettua uno shift a sinistra di un bit, aggiungendo uno zero come bit meno significativo (moltiplicazione per due)
 - Estende il segno a 32 bit



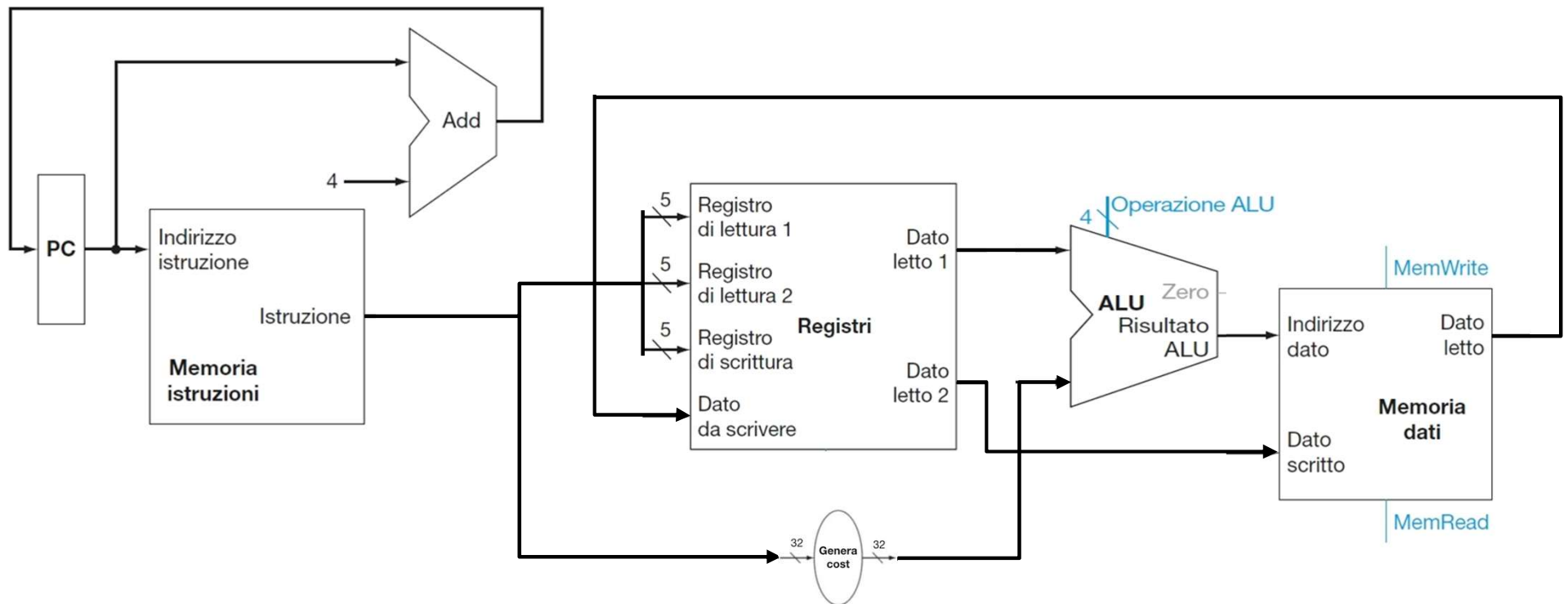
Unità di Elaborazione Unificata

- Abbiamo prodotto unità diverse per ogni tipo di istruzione (Tipo R)



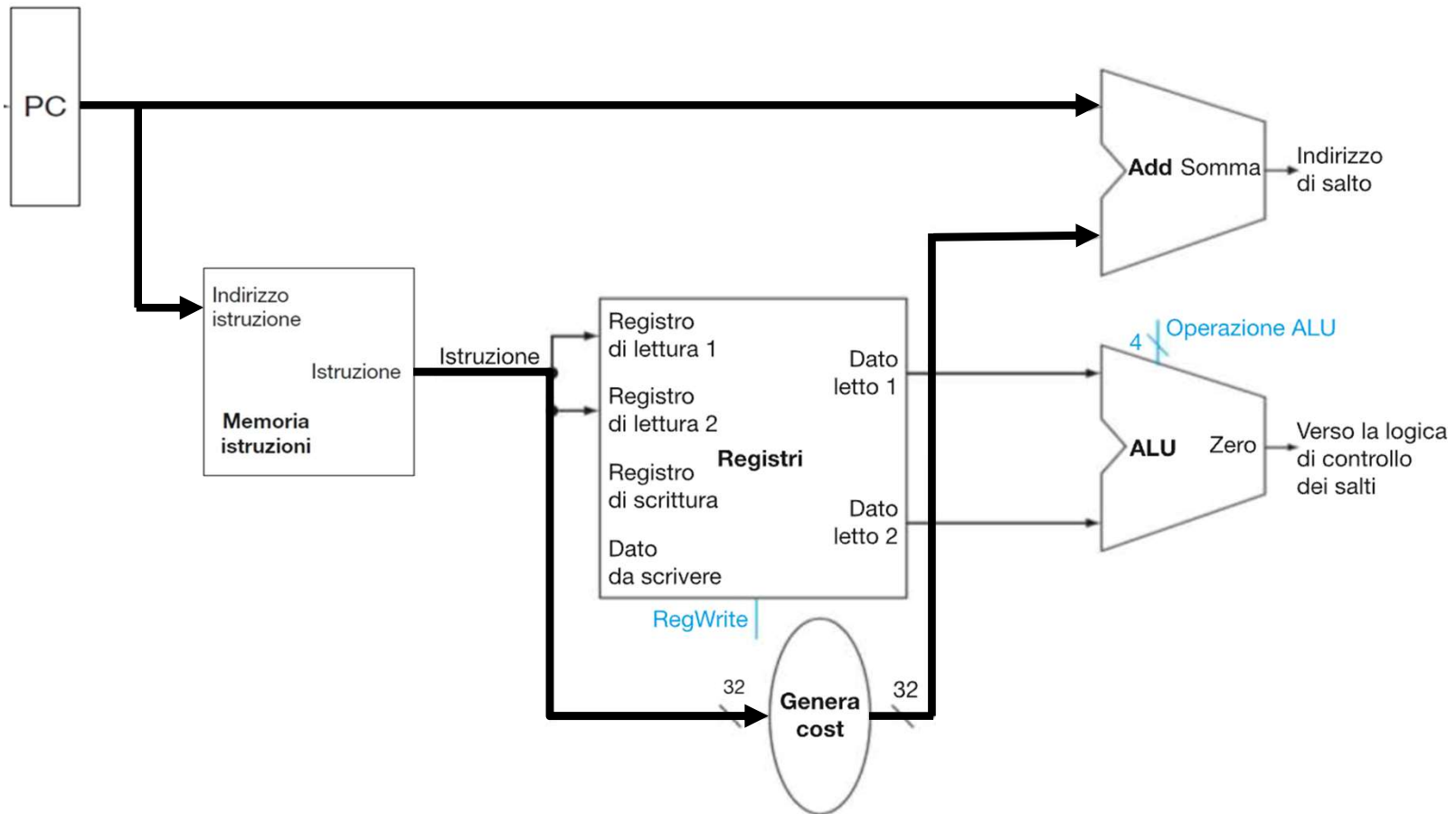
Unità di Elaborazione Unificata

- Abbiamo prodotto unità diverse per ogni tipo di istruzione (Load & Store)



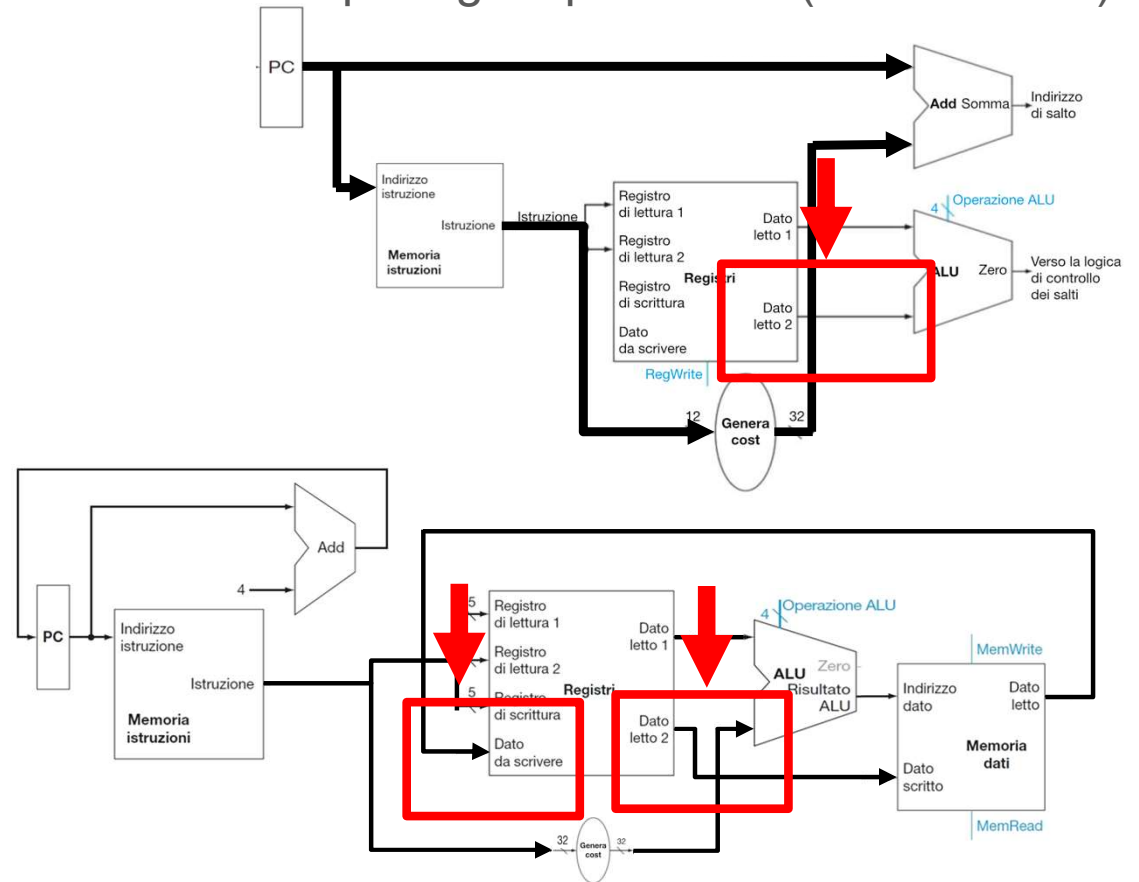
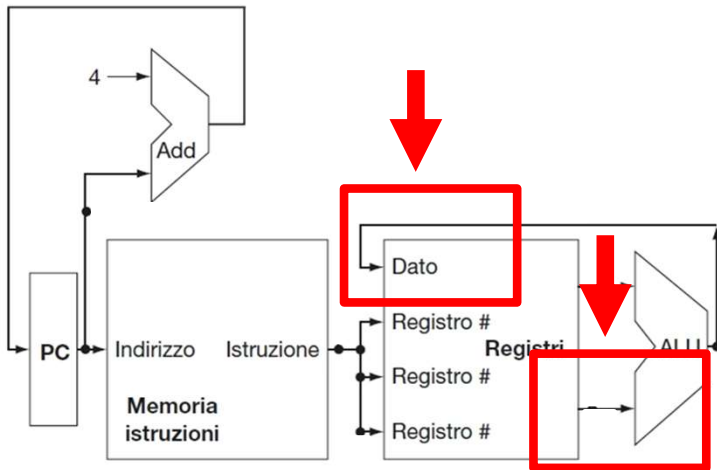
Unità di Elaborazione Unificata

- Abbiamo prodotto unità diverse per ogni tipo di istruzione (beq – Tipo SB)

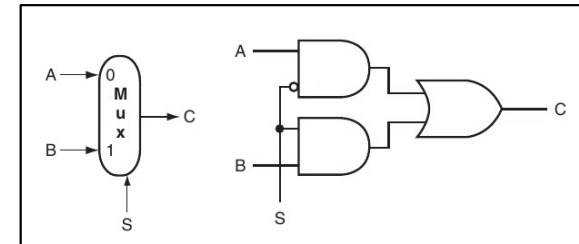
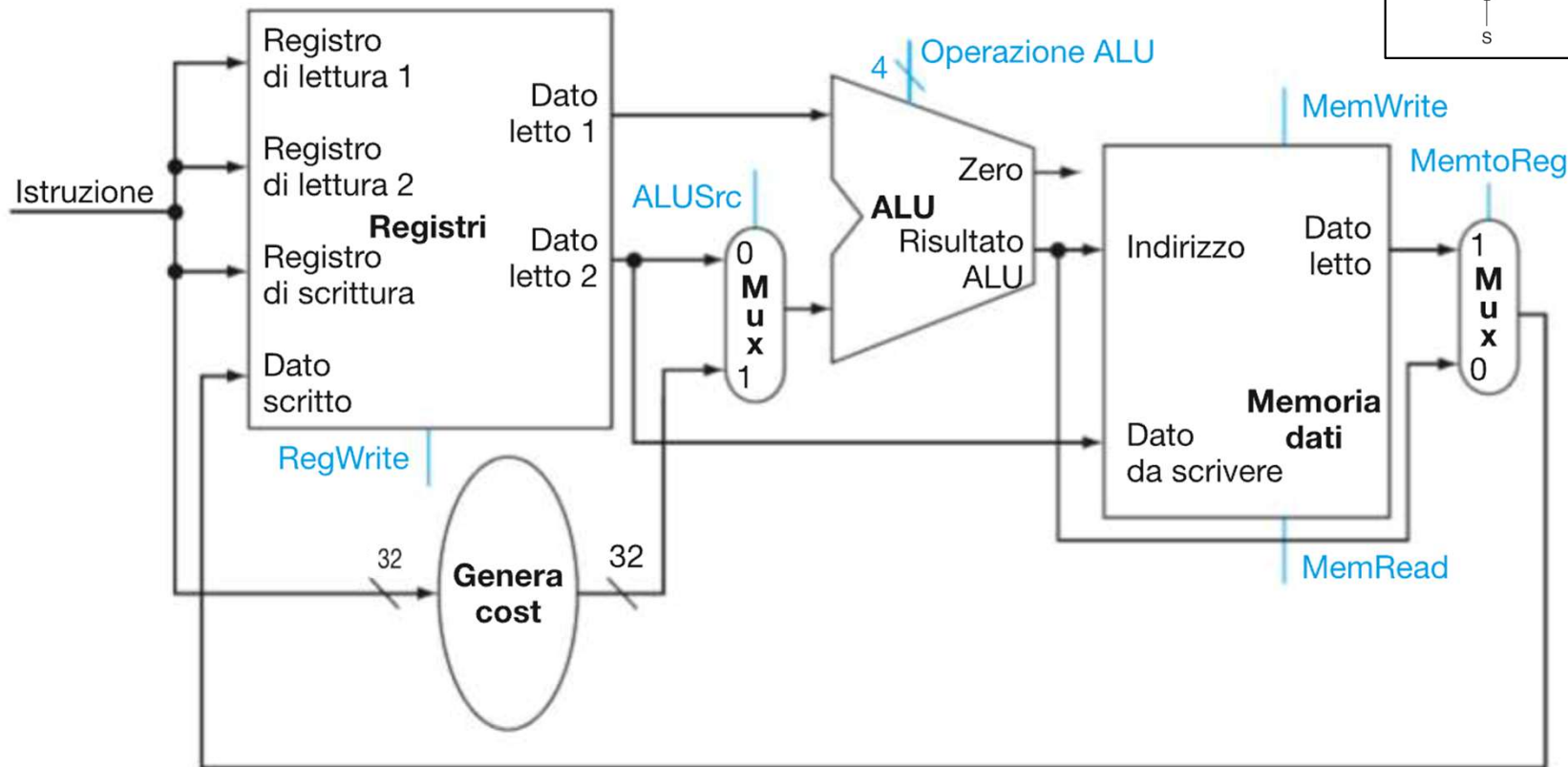


Unità di Elaborazione Unificata

- Come unificare le tre in un singolo processore RISC-V?
- Problema: Selezionare quale segnale considerare per ogni operazione (ciclo di clock)
- Soluzione: Multiplexers



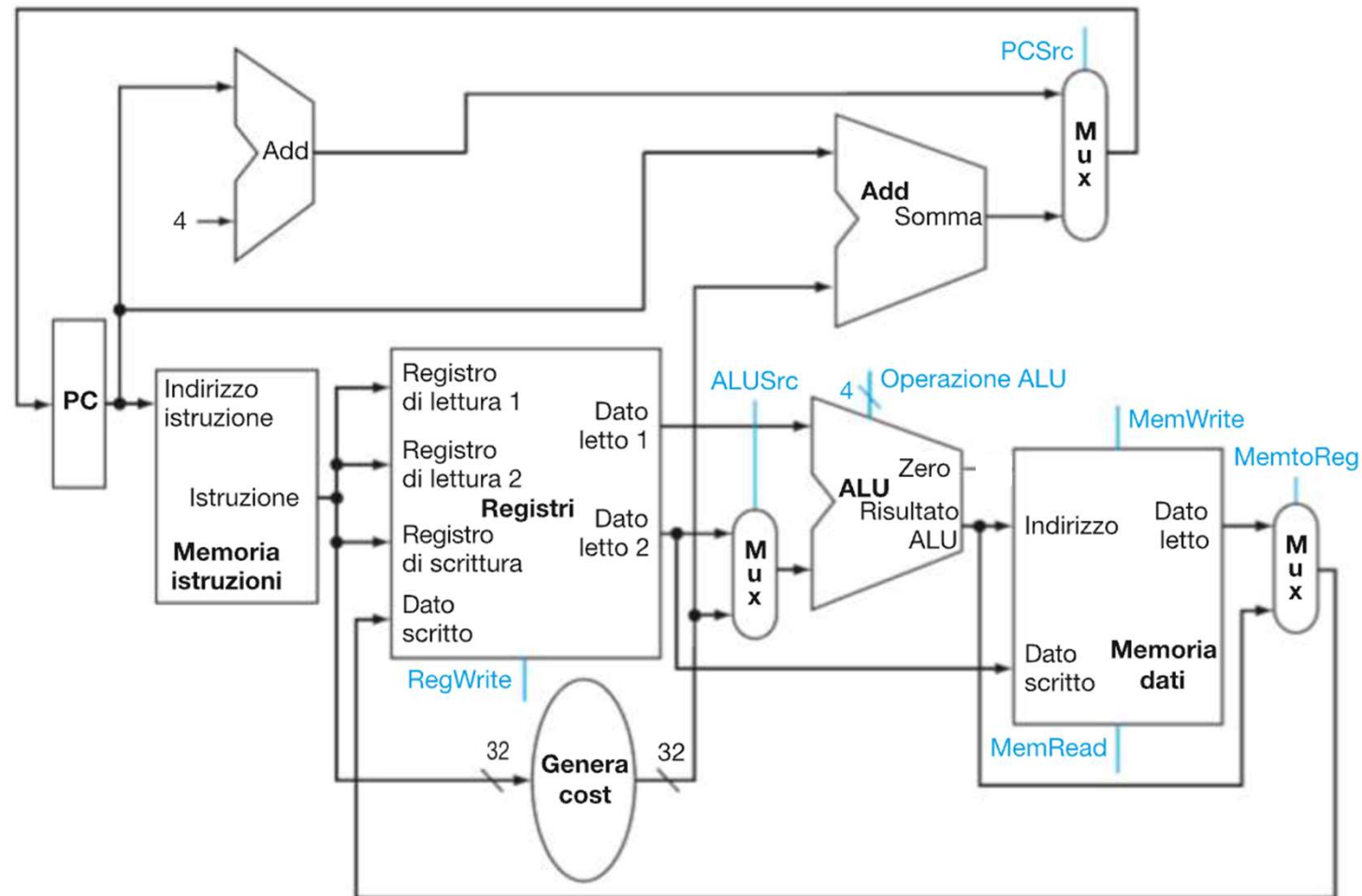
Unità di Elaborazione Unificata



Unità di elaborazione per le istruzioni di accesso alla memoria e per le istruzioni di tipo R, ottenuta con l'aggiunta di due multiplexer.

Unità di Elaborazione Unificata

- Unità di elaborazione per eseguire le istruzioni di base in un singolo ciclo di clock
- **Per integrare i salti condizionati si è resa necessaria l'aggiunta di un altro multiplexer**



Unità di Elaborazione Unificata

- Questa unità di elaborazione **esegue tutte le istruzioni in un singolo ciclo di clock.**
- Nessuna risorsa dell'unità potrà essere utilizzata più di una volta per ogni istruzione
- **Il controllo è fatto da un circuito combinatorio**
- Abbiamo duplicato diversi componenti (e.g., una memoria delle istruzioni separata da quella dei dati)

